

Paper visione classica: PatchMatch

Introduzione

Il paper **PatchMatch** presenta strumenti di **image editing interattivo** basati su un algoritmo **randomizzato** capace di trovare rapidamente corrispondenze tra **patch**, formulate come problemi di **Approximate Nearest Neighbors (ANN)**.

L'osservazione di partenza è che molte tecniche di grafica **patch-based** producono risultati di alta qualità, ma risultano computazionalmente costose perché richiedono, per ogni patch, la ricerca della patch più simile all'interno di un'altra immagine o regione. Con **PatchMatch**, questa operazione viene accelerata a tal punto da diventare quasi **interattiva**.

Le intuizioni fondamentali alla base dell'algoritmo sono due:

1. **Random sampling**: in uno spazio di ricerca molto ampio, un numero sufficientemente grande di tentativi casuali ha una buona probabilità di includere corrispondenze di qualità elevata.
nota: lo spazio delle patch è ad alta dimensionalità, ma PatchMatch evita di indicizzarlo con strutture complesse e usa campionamenti casuali guidati dalla soluzione corrente.
2. **Coerenza naturale delle immagini**: se un certo offset è una buona corrispondenza per una patch, è probabile che lo sia anche per patch spazialmente adiacenti. Questa regolarità consente di **propagare** informazioni utili tra vicini e migliorare rapidamente il campo delle corrispondenze

Oggi esistono algoritmi in grado di eseguire operazioni di image editing ad alto livello che vanno oltre il ritocco locale e agiscono sulla struttura dell'immagine. Tra queste operazioni rientrano:

- **Image retargeting**: ridimensionamento content-aware che preserva meglio contenuto semantico e struttura
- **Image completion**: ricostruzione plausibile di una regione rimossa a partire dal contesto
- **Image reshuffling**: riorganizzazione di regioni dell'immagine con sintesi coerente delle parti mancanti.

In questo contesto, l'interattività è fondamentale poiché l'utente spesso esplora diverse soluzioni per tentativi successivi, e quindi servono metodi che siano al tempo stesso:

- **flessibili**, cioè adattabili a operazioni diverse
- **veloci**, cioè capaci di mostrare rapidamente risultati intermedi

Molti approcci precedenti riuscivano a soddisfare solo uno di questi due requisiti. I metodi patch-based preservavano bene texture e strutture, ma diventavano lenti su immagini di dimensione medio grande.

Prima di PatchMatch

Prima di PatchMatch, il problema della corrispondenza tra patch compare in più filoni: nella **texture synthesis non parametrica**, nell'**inpainting exemplar-based** e nei metodi globali di **retargeting** e **completion**.

Un primo riferimento importante è la **sintesi non parametrica** di *Efros & Leung (1999)*, in cui la texture viene generata un pixel alla volta a partire da un piccolo seed iniziale. Per ogni nuovo pixel, il metodo cerca nel campione i neighborhood più simili a quello già osservabile attorno alla posizione corrente e usa tali corrispondenze per assegnarne il valore, assumendo quindi che la texture sia governata soprattutto da

dipendenze locali. L'idea è molto elegante e influente, ma il costo resta elevato perché richiede una ricerca estesa quasi per ogni pixel sintetizzato.

Un passo successivo è **image quilting** di *Efros & Freeman (2001)*: invece di sintetizzare un pixel alla volta, il metodo costruisce l'immagine copiando piccole patch dal campione e facendole combaciare nelle zone di overlap. Le discontinuità tra patch adiacenti vengono ridotte scegliendo un seam a costo minimo lungo la regione sovrapposta, così da ottenere risultati più stabili ed efficienti rispetto alla sintesi pixel-wise. Anche in questo caso, però, il nucleo costoso del problema rimane la ricerca di patch compatibili e visivamente plausibili.

Nei problemi di **completion**, la stessa difficoltà si ripresenta in metodi come quello di *Wexler et al. (2007)*, che formulano il completamento come un problema di ottimizzazione globale su patch spazio-temporali. L'idea è che ogni patch dentro e attorno alla regione mancante debba trovare una controparte simile nei dati disponibili, ma anche che tutte queste scelte locali risultino tra loro coerenti nello spazio e nel tempo. Per questo il metodo alterna una fase di stima delle corrispondenze tra patch e una fase di ricostruzione dell'output tramite aggregazione pesata dei candidati, così da evitare le incoerenze tipiche dei metodi puramente greedy o locali.

In modo simile, la **bidirectional similarity** di *Simakov et al. (2008)* descrive il retargeting come un problema globale basato su patch a più scale, in cui il target deve essere al tempo stesso *completo* rispetto alla sorgente (preservare più informazione possibile) e *coerente* con essa (non introdurre patch o artefatti non presenti nell'input). Questo richiede quindi corrispondenze dense e affidabili tra patch nelle due direzioni, dalla sorgente al target e dal target alla sorgente.

In tutti questi casi il collo di bottiglia resta lo stesso: per molte patch bisogna trovare rapidamente corrispondenze locali plausibili in uno spazio di ricerca molto ampio. È proprio su questo punto che PatchMatch introduce il suo contributo principale, cioè una stima approssimata ma estremamente efficiente del **nearest-neighbor field**.

Costi computazionali

La lentezza deriva da un'operazione comune a tutta questa famiglia di metodi: la ricerca, per ogni patch, della patch più simile in un'altra regione o immagine.

Sia m il numero di pixel in una patch (se la patch è $p \times p$, allora $m = p^2$).

Sia M il numero di posizioni patch valide in una regione o immagine.

Per ogni patch in una regione A (circa M patch) si confrontano tutte le patch candidate in una regione B (circa M patch), e ogni confronto costa $O(m)$. La corrispondenza risultante, che associa a ogni posizione in A la migliore posizione in B , è detta **Nearest-Neighbor Field (NNF)**.

Un approccio brute force ha quindi costo:

$$O(mM^2)$$

che diventa rapidamente proibitivo su immagini grandi.

Anche tecniche classiche di accelerazione basate su **approximate nearest neighbors** e **riduzione di dimensionalità** non eliminano completamente il collo di bottiglia della ricerca tra patch nei metodi patch-based.

Nel paper, gli autori osservano che strutture come i **kd-tree** operano nello spazio ad alta dimensionalità delle patch, mentre il loro metodo lavora direttamente nello spazio bidimensionale degli **offset**. Inoltre, le strutture ad albero richiedono memoria aggiuntiva dell'ordine di $O(M)$ o superiore, con costanti relativamente grandi, il che ne limita l'applicazione su immagini ad alta risoluzione. PatchMatch cambia

quindi prospettiva: invece di concentrare la ricerca nello spazio delle patch, sfrutta la **coerenza naturale delle immagini** e una ricerca randomizzata nello spazio 2D degli offset, ottenendo una maggiore efficienza pratica e una minore richiesta di memoria.

Prestazioni

Gli autori mostrano che PatchMatch ha buone proprietà di convergenza teoriche su un caso sintetico e ottime prestazioni empiriche su immagini reali, risultando efficace anche su immagini fino a **2 megapixel**.

Nel lavoro originale, l'implementazione CPU mostra accelerazioni significative rispetto a metodi ANN basati su kd-tree con PCA, mentre una versione GPU introduce ulteriori speedup. Più che su una garanzia asintotica forte, il vantaggio del metodo si osserva nella pratica: il costo per iterazione è contenuto, la memoria richiesta è lineare nel numero di patch, e poche iterazioni bastano spesso per ottenere un NNF approssimato di buona qualità.

In questo senso, PatchMatch affronta direttamente il collo di bottiglia dei metodi patch based dato che calcola un NNF approssimato in modo molto efficiente, rendendo praticabili in tempi rapidi numerose applicazioni di editing strutturale.

Algoritmo PatchMatch per il calcolo del Nearest Neighbor Field

Dato un dominio di patch nell'immagine A e un dominio di patch nell'immagine B , l'obiettivo è associare a ogni patch di A la patch più simile in B , secondo una certa distanza tra patch.

Il **Nearest-Neighbor Field** viene definito formalmente come una funzione:

$$f : A \rightarrow \mathbb{R}^2$$

che, per ogni coordinata di patch a in A , restituisce un offset bidimensionale $f(a)$ tale che la patch centrata in a venga confrontata con la patch centrata in $a + f(a)$ in B .

Se b è la posizione della patch più simile in B , allora:

$$f(a) = b - a.$$

Il NNF è quindi un campo di vettori di traslazione 2D memorizzato su tutte le patch di A .

PatchMatch si basa su tre osservazioni:

- invece di indicizzare lo spazio ad alta dimensionalità delle patch, lavora sullo spazio 2D degli offset
- patch adiacenti in A tendono ad avere offset simili verso B
- una inizializzazione casuale ha buone probabilità di produrre alcuni seed utili che possono poi essere propagati.

L'algoritmo parte da una inizializzazione del campo di offset e procede iterativamente alternando due fasi: **propagazione** e **random search**.

Distanza tra patch $D(\cdot, \cdot)$

La ricerca delle corrispondenze tra patch è definita rispetto a una funzione di distanza D tra patch. Nel paper, il problema viene introdotto in termini generali, osservando che il nearest neighbor può essere cercato secondo una metrica tra patch, ad esempio una metrica di tipo L_p .

Nella formulazione della bidirectional similarity utilizzata per gli strumenti di editing, la distanza adottata è invece la **SSD (Sum of Squared Differences)** calcolata sui valori dei pixel nello spazio colore $L \cdot a \cdot b$. Il contributo principale del lavoro non consiste nell'introdurre nuove metriche di confronto tra patch, ma nel

rendere molto più efficiente il calcolo approssimato del Nearest-Neighbor Field mediante inizializzazione, propagazione e random search nello spazio degli offset.

Inizializzazione del campo di offset

Il **Nearest-Neighbor Field** può essere inizializzato in due modi: mediante **offset casuali** oppure sfruttando una stima precedente. Nel caso dell'inizializzazione casuale, a ogni posizione di patch in A viene associato un offset estratto uniformemente nell'intero dominio valido di B .

Nelle applicazioni descritte nel paper, gli autori impiegano anche una procedura **coarse-to-fine** su piramide multi-risoluzione e possono quindi usare come stima iniziale un campo di offset ottenuto per upsampling dal livello precedente.

Tuttavia, il paper sottolinea che un uso esclusivo di tale prior può intrappolare l'algoritmo in minimi locali subottimali. Per questo motivo, gli autori eseguono inizialmente alcune iterazioni con inizializzazione casuale, poi confrontano questa soluzione con quella upscalata dal livello precedente e mantengono, per ogni patch, l'offset associato alla distanza D minore.

Iterazione

Dopo l'inizializzazione, l'algoritmo esegue un numero ridotto di iterazioni, tipicamente 4 o 5. Gli autori mostrano teoricamente su un caso sintetico ed empiricamente su immagini reali che poche iterazioni sono spesso sufficienti per ottenere un NNF approssimato di buona qualità.

1) Propagazione

La fase di propagazione mira a migliorare il match della patch centrata in (x, y) sfruttando gli offset dei vicini già visitati secondo l'ordine di scansione.

Nel caso di una scansione da sinistra a destra e dall'alto in basso, si assume che patch adiacenti abbiano spesso corrispondenze simili; per questo motivo, per aggiornare $f(x, y)$ si confrontano tre candidati:

- l'offset corrente $f(x, y)$;
- l'offset del vicino sinistro $f(x - 1, y)$;
- l'offset del vicino superiore $f(x, y - 1)$.

Se $D(v)$ denota la distanza tra la patch in A centrata in (x, y) e la patch in B centrata in $(x, y) + v$, allora l'aggiornamento può essere scritto come

$$f(x, y) = \arg \min_{v \in \mathcal{C}(x, y)} D(v),$$

dove

$$\mathcal{C}(x, y) = \{f(x, y), f(x - 1, y), f(x, y - 1)\}.$$

L'effetto è che, se una patch corretta viene trovata in una regione coerente, essa può propagarsi rapidamente alle patch vicine lungo la direzione di scansione. Nelle iterazioni successive, invertendo il verso di scansione, la propagazione avviene anche nelle direzioni opposte.

2) Random search

La propagazione è un meccanismo locale e serve quindi una fase che consenta salti più ampi e permetta di uscire da minimi locali.

A partire dall'offset corrente $v_0 = f(x, y)$, si testano offset casuali in una finestra di raggio decrescente:

$$u_i = v_0 + w\alpha^i R_i$$

dove:

- R_i è un vettore casuale uniforme in $[-1, 1] \times [-1, 1]$
- w è il raggio iniziale
- $\alpha \in (0, 1)$ è il fattore di riduzione del raggio

Si continua finché il raggio scende sotto 1 pixel.

Efficienza

Due ottimizzazioni pratiche importanti sono:

- **early termination** nel calcolo della distanza: se la somma parziale supera già il best corrente, si interrompe il confronto
- **aggiornamenti incrementali**: sfruttando la sovrapposizione tra patch vicine, si può evitare di ricomputare interamente ogni distanza.

Nel paper viene inoltre proposta una variante GPU che approssima la propagazione seriale tramite meccanismi più parallelizzabili.

Analisi di convergenza su caso sintetico e comportamento su immagini reali

Nel caso sintetico discusso dagli autori, si considera una regione distintiva R immersa in un contesto altrimenti uniforme. Per evitare ambiguità con la notazione usata in precedenza per la dimensione delle patch, indichiamo con r la dimensione della regione distintiva, con M il numero totale di pixel dell'immagine e con C l'ampiezza dell'intorno considerato sufficientemente vicino all'offset corretto. L'idea è che, se almeno una corrispondenza casuale cade entro un intorno corretto, allora la fase di propagazione può diffondere rapidamente tale soluzione all'intera regione coerente.

In questo modello, la probabilità che almeno uno dei campioni casuali atterri nell'intorno corretto è:

$$p = 1 - \left(1 - \frac{C}{M}\right)^m$$

Il numero atteso di iterazioni prima della convergenza segue quindi una distribuzione geometrica ed è dato da:

$$\langle t \rangle = \frac{1}{p} - 1$$

Introducendo inoltre il rapporto relativo $\gamma = \frac{m}{M}$, nel limite di immagini grandi si ottiene:

$$\lim_{M \rightarrow \infty} \langle t \rangle = [1 - \exp(-C\gamma)]^{-1} - 1$$

Questa analisi teorica supporta l'idea che, nel caso sintetico considerato, l'algoritmo converga con alta probabilità in poche iterazioni. Su immagini reali, gli autori mostrano inoltre empiricamente che l'**NNF approssimato** raggiunge rapidamente una buona qualità pratica, tipicamente già dopo 4 o 5 iterazioni.

Editing tools

Una volta disponibile un NNF approssimato in tempi ridotti, lo stesso motore di ricerca può essere utilizzato in diversi strumenti di **structural image editing**. In questa parte del paper, gli autori riprendono l'approccio di **bidirectional similarity** introdotto da Simakov et al. per definire una misura di distanza tra immagine sorgente S e immagine target T . Tale misura combina due componenti: un termine di **completezza**, che richiede che il target contenga quanta più informazione visiva possibile dalla sorgente,

e un termine di **coerenza**, che penalizza l'introduzione nel target di strutture non supportate dall'immagine di input.

Formalmente, la distanza è definita come:

$$d_{BDS}(S, T) = \underbrace{\frac{1}{N_S} \sum_{s \in S} \min_{t \in T} D(s, t)}_{d_{\text{complete}}(S, T)} + \underbrace{\frac{1}{N_T} \sum_{t \in T} \min_{s \in S} D(t, s)}_{d_{\text{cohere}}(S, T)}$$

dove D è la **SSD** calcolata sui valori delle patch nello spazio colore $L \cdot a \cdot b$.

L'ottimizzazione dell'immagine target avviene in modo iterativo. Data una stima iniziale T_0 , ogni iterazione segue uno schema di tipo **EM**: nell'**E-step** si calcolano i nearest-neighbor fields tra sorgente e target corrente e si esegue il **patch voting**; nell'**M-step** i voti di colore vengono **mediati** per ottenere la nuova immagine T_{i+1} . Per ridurre il rischio di minimi locali, il processo è inserito in una procedura multi-scala di **gradual scaling**, in cui il target viene inizializzato a bassa risoluzione e progressivamente ridimensionato fino alla dimensione finale.

Questo schema consente di costruire strumenti per image completion, retargeting e reshuffling. In particolare:

- nell'**image completion** il sistema può essere guidato anche da vincoli interattivi, ad esempio linee o etichette che indicano come certe strutture debbano proseguire all'interno della regione mancante
- nell'**image retargeting** l'approccio patch-based consente di preservare meglio linee architettoniche, strutture ripetitive e regioni visivamente importanti
- nell'**image reshuffling** l'utente può spostare oggetti o porzioni della scena e lasciare al sistema il compito di ricostruire in modo coerente le aree residue

Vincoli geometrici e limiti del metodo

Un contributo importante del paper è l'introduzione di **vincoli geometrici** nel processo di sintesi. Gli autori formalizzano il problema come una minimizzazione vincolata:

$$\arg \min d_{BDS} \quad \text{s.t.} \quad M_k(p_i, NN(p_i), q_j, NN(q_j)) = 0$$

cioè la bidirectional similarity viene minimizzata imponendo che alcune corrispondenze rispettino anche modelli geometrici desiderati.

Nel caso dei **vincoli su linee**, si può imporre che i punti corrispondenti appartengano a una stessa retta:

$$NN(p_i)^T l = 0$$

dove l è il vettore della retta in coordinate omogenee.

Nel caso dei **vincoli su regioni**, si può invece usare una trasformazione omografica H_k :

$$H_k p_i - NN(p_i) = 0$$

forzando così certe regioni a trasformarsi secondo modelli controllati, ad esempio traslazioni, cambi di scala o trasformazioni prospettiche. La stima di questi modelli viene resa robusta tramite **RANSAC**, così da limitare l'effetto di rumore e outlier.

Verso la fine del lavoro, gli autori discutono anche i limiti del metodo. In presenza di:

- input patologici
- modifiche molto estreme

- strutture fortemente ambigue

possono presentarsi **failure cases**, ad esempio:

- **ghosting**
- **feathering**
- convergenza verso minimi locali non soddisfacenti

Tuttavia, la grande velocità di PatchMatch costituisce anche un vantaggio operativo: l'utente può aggiungere vincoli, modificare l'inizializzazione e rilanciare il processo più volte, esplorando rapidamente diverse soluzioni.

Conclusione

PatchMatch rappresenta un contributo fondamentale nei metodi patch-based, mostrando come la ricerca approssimata dei nearest neighbor possa essere resa efficiente sfruttando la coerenza spaziale e una ricerca randomizzata.

La sua importanza non sta solo nell'accuratezza del matching, ma soprattutto nell'aver trasformato un collo di bottiglia computazionale in una procedura abbastanza veloce da rendere praticabili applicazioni interattive di image editing strutturale.

Progetto: implementazione e valutazione di PatchMatch

Il progetto implementa il calcolo del **Nearest-Neighbor Field (NNF)** e ne dimostra il comportamento tramite esperimenti quantitativi e qualitativi, includendo anche un'estensione per **inpainting multi-scala** in stile *Search + Vote*.

L'idea centrale è separare chiaramente:

- il **core algoritmico** di PatchMatch, cioè la stima approssimata di corrispondenze tra patch di due immagini
- alcune **applicazioni e analisi sperimentali**, tra cui ricostruzione, studio della convergenza, confronto con brute force, toy example passo-passo, failure cases, inpainting e confronto con un metodo deep

Codici originali repository

repo: <https://github.com/MingtaoGuo/PatchMatch/tree/master>

PatchMatch.py

Il codice implementa una versione basica di PatchMatch per stimare un campo di corrispondenze tra due immagini. Per ogni pixel o centro-patch dell'immagine **A** viene cercata una patch simile in **B** tramite:

- **inizializzazione casuale** del campo f (NNF) e della mappa delle distanze `dist`
- **propagazione** in scanline order (dispari) e reverse order (pari)
- **random search** con finestre decrescenti
- **ricostruzione** finale copiando, per ogni pixel di A, il valore di B nella posizione indicata dal NNF

Nota su codice originale: nella versione base, la propagazione seleziona direttamente il match del vicino senza applicare in modo esplicito lo shift coerente con l'idea teorica di propagare il candidato del vicino alla posizione corrente. Inoltre, il random search parte con finestre già relativamente piccole rispetto alla formulazione standard del paper.

PatchMatch_Bidirectional.py

Il codice contiene una variante che:

- usa una distanza patch che include anche canali o feature aggiuntive
- normalizza globalmente gli input prima del matching
- implementa una propagazione con shift ± 1 , più vicina alla descrizione del paper

Elementi presi dai codici originali

- **Schema complessivo PatchMatch:** inizializzazione casuale + iterazioni alternate (scanline/reverse) composte da propagazione e random search.
- **Gestione dei bordi tramite padding e mascheramento:** in `PatchMatch.py` il confronto ignora i NaN ; la stessa idea viene mantenuta nella nuova distanza con `np.isnanum` e conteggio dei pixel validi.
- **Propagazione con shift:** la logica “shifted ± 1 ” coerente col paper è presente nel codice bidirezionale ed è stata ripresa nella nuova implementazione.

Modifiche principali

- **Rappresentazione dati più pulita:** NNF come array `(H,W,2) int32` e NND come array `float64`, evitando `dtype=object` del codice base.
- **Random search allineato al paper:** finestra che si restringe esponenzialmente con fattore `alpha`, partendo da un raggio massimo pari alla dimensione dell'immagine.
- **Visualizzazione NNF in HSV** (hue = direzione, value = magnitudine), dichiarata come analoga alla rappresentazione intuitiva del campo di offset.
- **Ricostruzione in due modalità:** ricostruzione **pixel-wise** e ricostruzione tramite **media di patch sovrapposte**, utile per ottenere risultati più stabili e meno rumorosi.
- **Storico dell'evoluzione dell'algoritmo:** funzione `patchmatch_history`, utile per visualizzare iterazione per iterazione NNF, RMS map e ricostruzione.
- **Estensione “Inpainting multi-scale Search+Vote”:** implementazione di una pipeline completa (piramide, inizializzazione, PatchMatch vincolato, voto) che non era presente nei codici originali.

Inpainting multi-scala

L'inpainting viene eseguito tramite una pipeline coarse-to-fine:

- costruzione di una piramide multi-scala di immagine e maschera
- inizializzazione grezza del buco
- iterazioni tipo EM che alternano una fase **Search** (PatchMatch vincolato a patch sorgenti completamente note) e una fase **Vote** (ricostruzione dei pixel mancanti tramite aggregazione di patch sovrapposte)

Più in dettaglio:

- la sorgente valida viene vincolata a patch interamente contenute nella regione nota
- la distanza usata in inpainting è una **MSE pesata da una confidence map**
- alle scale più fini il contenuto mancante viene inizializzato tramite **upsampling** del risultato della scala precedente

demo.py e analisi dei risultati

Qui vengono eseguiti i seguenti esperimenti:

- **1:** NNF + NNF visualization + reconstruction
- **2:** patchmatch convergence (RMS per iteration)
- **3:** multiscale inpainting search + vote
- **4:** Root Mean Square (RMS) - patchmatch vs ground truth
- **5:** effect of patch size on quality and time
- **6:** toy example step-by-step
- **7:** failure cases
- **8:** patchMatch vs LaMa
- **9:** applications

Metodologia comune: metrica RMS

La funzione `rms_from_nnd` definisce RMS come:

$$\text{RMS} = \sqrt{\max(\text{mean}(\text{NND}), 0)}$$

dove NND è la mappa delle MSE normalizzate.

La RMS viene usata come misura sintetica della qualità media del matching: valori più bassi indicano corrispondenze migliori.

Esperimento 1 - NNF + visualizzazione NNF + ricostruzione

L'esperimento esegue una singola stima del **Nearest-Neighbor Field** tra l'immagine sorgente **A** e la target **B**, ottenendo in output il campo `nnf` e la mappa delle distanze `nnd`.

La visualizzazione finale comprende:

- immagine sorgente **A**
- immagine target **B**
- **NNF visualization** in HSV
- **ricostruzione pixel-wise**
- **ricostruzione da patch sovrapposte**
- **RMS map**



In alto: immagine sorgente A, immagine target B e visualizzazione del Nearest-Neighbor Field (NNF) in HSV (hue = direzione dell'offset, value = magnitudine). In basso: ricostruzione pixel-wise, ricostruzione ottenuta mediando patch sovrapposte e mappa locale dell'errore RMS.

Risultati osservati

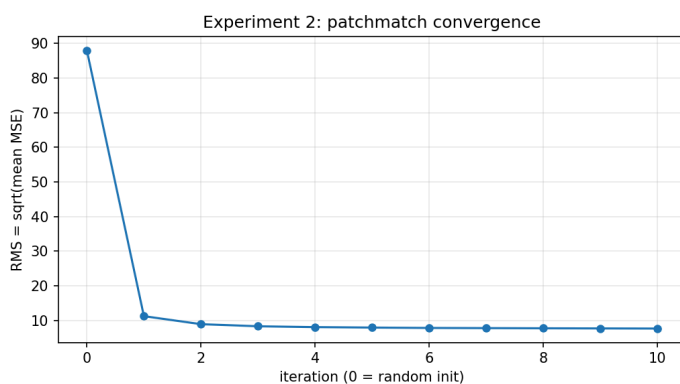
- time = 62.02s , RMS = 8.0839

Il valore RMS indica l'errore medio tra patch di A e patch corrispondenti in B secondo l'NNF stimato: un RMS più basso implica corrispondenze mediamente migliori rispetto all'inizializzazione casuale.

La ricostruzione **pixel-wise** è utile come diagnostica, ma può produrre artefatti locali perché copia un solo pixel per posizione. La ricostruzione **patch-based** tende invece a essere più stabile, perché combina contributi di patch sovrapposte. In questo senso, l'esperimento evidenzia bene la differenza tra semplice uso del NNF come mappa di corrispondenze e uso del matching come base per una vera ricostruzione.

Esperimento 2 - Convergenza PatchMatch (RMS per iterazione)

L'esperimento misura la **convergenza** dell'algoritmo registrando la RMS a ogni iterazione. Per rendere i risultati riproducibili, si parte da una inizializzazione fissata.



Curva della RMS in funzione delle iterazioni: l'iterazione 0 corrisponde all'inizializzazione casuale del NNF,

mentre i punti successivi mostrano il miglioramento progressivo dovuto a propagazione e random search, con rapido calo iniziale e successivo assestamento.

Risultati osservati

- iter 0: RMS = 87.9349
- iter 1: 11.2674
- iter 2: 8.9630
- iter 3: 8.3566
- iter 10: 7.7034

La discesa molto rapida dall'iterazione 0 all'iterazione 1 è il comportamento atteso: l'inizializzazione casuale è molto rumorosa, mentre già una singola iterazione di cooperazione locale (propagazione) + esplorazione (random search) migliora drasticamente la qualità del matching.

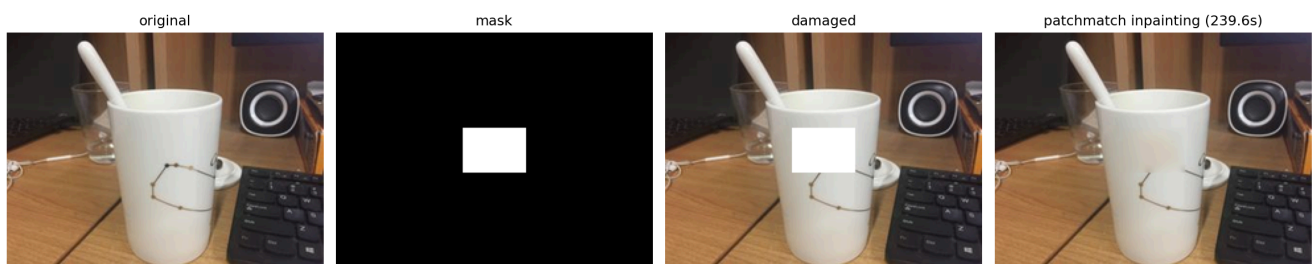
Dopo poche iterazioni la curva tende ad appiattirsi. Questo è coerente con l'idea del paper secondo cui poche iterazioni sono spesso sufficienti per una convergenza pratica.

Esperimento 3 - Inpainting multi-scala Search+Vote

L'esperimento costruisce un caso di **inpainting** introducendo una maschera rettangolare su A e generando un'immagine danneggiata in cui i pixel nel buco vengono neutralizzati. L'inpainting viene poi eseguito con una pipeline coarse-to-fine:

- costruzione di una piramide multi-scala dell'immagine e della maschera
- inizializzazione del buco con un riempimento grezzo
- iterazioni tipo EM in cui si alternano una fase **Search** (PatchMatch vincolato a patch sorgenti completamente note) e una fase **Vote** (ricostruzione dei pixel del buco tramite aggregazione di patch sovrapposte)

Experiment 3: multiscale inpainting, ps=5



Confronto tra immagine originale, maschera, immagine danneggiata con hole rettangolare e risultato finale dell'inpainting.

Risultati osservati

Tre scale: $60 \times 44 \rightarrow 120 \times 89 \rightarrow 240 \times 179$.

- A scale 0 la RMS scende da 15.3950 a 8.7005
 - A scale 1 scende da 6.3000 a 5.2321
 - A scale 2 oscilla ma resta nell'intorno 4.1-4.7, terminando a 4.3738
- Tempo totale: 242.54s

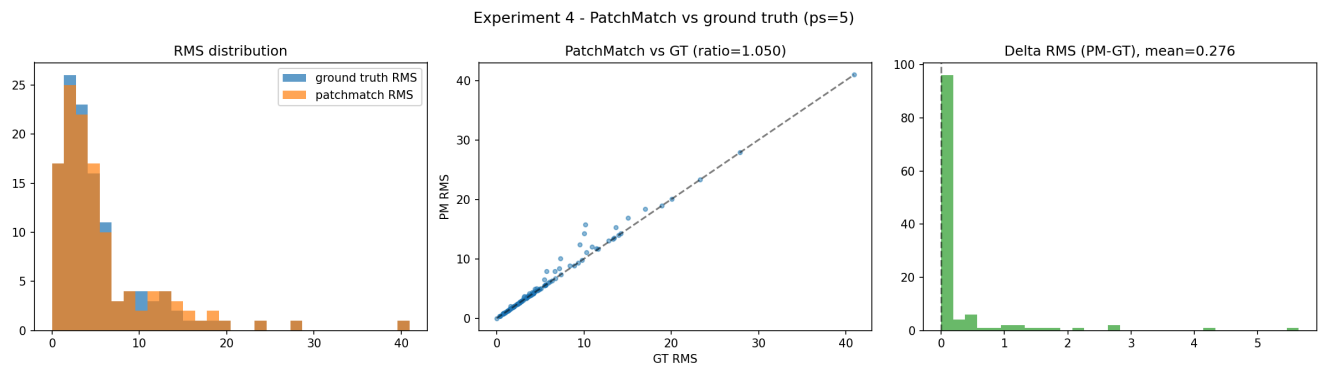
Il comportamento multi-scala è coerente con la strategia coarse-to-fine: a bassa risoluzione si impone una struttura globale plausibile, poi alle scale maggiori si rifiniscono texture e dettagli.

Le oscillazioni a scala piena sono compatibili con un ciclo iterativo “Search+Vote”: il voto modifica l'immagine corrente, e quindi modifica anche il paesaggio delle distanze al passo successivo. Non è quindi richiesta una monotonia perfetta dell'errore a ogni iterazione EM.

L'esperimento mostra bene come PatchMatch possa essere usato non solo per stimare corrispondenze, ma anche come motore di una pipeline di **completion/inpainting**.

Esperimento 4 - Confronto PatchMatch vs Ground Truth (brute force su subset)

L'esperimento fornisce una validazione quantitativa confrontando PatchMatch con una soluzione **esatta** calcolata tramite ricerca esaustiva su un sottoinsieme di punti query.



A sinistra: distribuzione della RMS per ground truth e PatchMatch. Al centro: scatter plot $RMS(PM)$ vs $RMS(GT)$ con retta $y = x$. A destra: distribuzione di $\Delta RMS = RMS(PM) - RMS(GT)$.

Risultati osservati

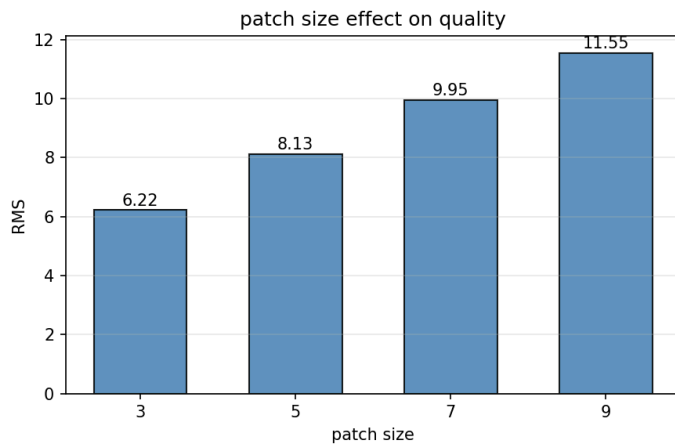
- 300 query (stride=10), brute force: 12.39M confronti
- tempi: PatchMatch $\approx$ 62.50s (6 iterazioni), GT $\approx$ 115.3s (subset)
- RMS mean: GT = 4.7821, PM = 5.0274, rapporto PM/GT = 1.051
- delta medio (PM-GT) = 0.2452
- violazioni PM < GT: 0/300

Ci si aspetta **PM $\geq$ GT**: la brute force calcola il vero minimo nel dominio dei candidati testati, mentre PatchMatch è approssimato. L'assenza di “violazioni” è quindi un controllo di coerenza importante.

Il rapporto 1.051 indica che, sul subset testato, PatchMatch produce risultati mediamente solo di circa il 5% peggiori dell'ottimo, a fronte di un tempo inferiore e di una procedura molto più scalabile di una brute force completa.

Esperimento 5 - Effetto della patch size su qualità (RMS) e tempo

L'esperimento analizza l'impatto della dimensione della patch ripetendo PatchMatch con diversi valori di `patch_size`, mantenendo fissi gli altri iperparametri principali.



Andamento della RMS al variare della dimensione della patch: patch più grandi impongono vincoli di somiglianza più stringenti e, nel caso in esame, producono un incremento dell'errore medio.



Confronto visivo tra l'immagine originale A e le ricostruzioni ottenute con diverse dimensioni di patch tramite media di patch sovrapposte. Ogni pannello riporta la RMS corrispondente.

Risultati osservati

- ps = 3, RMS = 6.2249, time $\approx$ 62.36s
- ps = 5, RMS = 8.1291, time $\approx$ 63.40s
- ps = 7, RMS = 9.9549, time $\approx$ 65.20s
- ps = 9, RMS = 11.5504, time $\approx$ 65.84s

Nel setup considerato, patch più grandi producono valori medi di RMS più elevati. Questo è coerente con il fatto che patch più grandi impongono un vincolo di somiglianza più rigido, rendendo più difficile trovare corrispondenze molto vicine.

Va però ricordato che, cambiando la patch size, cambia anche la nozione operativa di similarità tra patch. Per questo il confronto numerico tra RMS ottenute con patch diverse va letto con cautela e accompagnato dal confronto visivo delle ricostruzioni.

Il tempo cresce moderatamente nel range testato, in modo coerente con l'aumento del costo del calcolo della distanza all'aumentare del numero di pixel per patch.

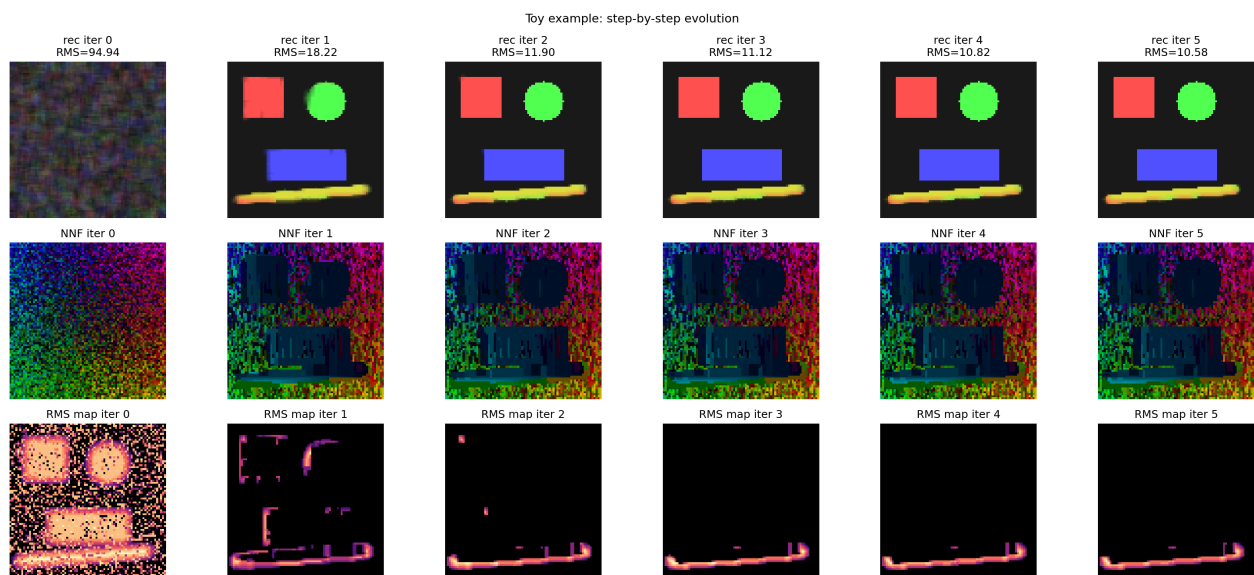
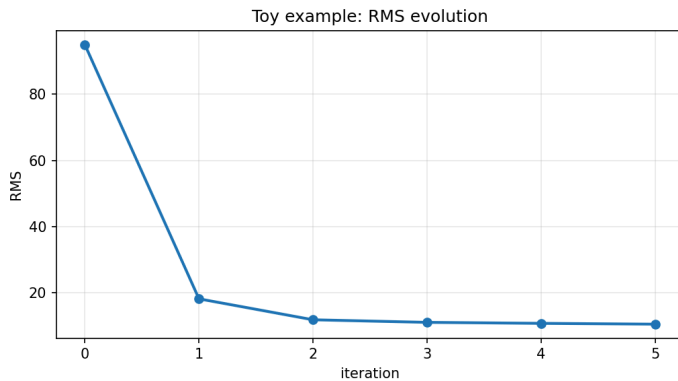
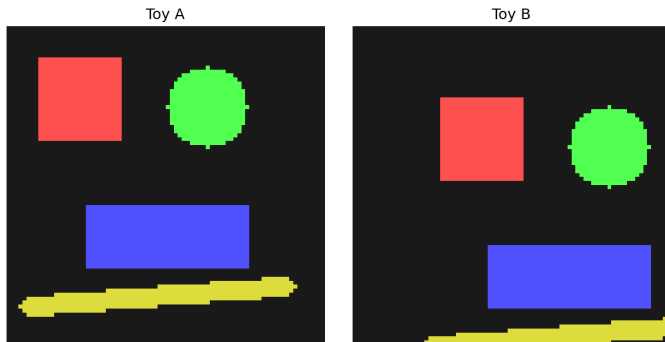
Esperimento 6 - Toy example: visualizzazione step by step

Questo esperimento introduce un **toy example sintetico** in cui l'immagine **B** è ottenuta traslando una configurazione semplice di forme colorate presente in **A**. L'obiettivo è visualizzare in modo chiaro l'evoluzione di PatchMatch su un caso controllato.

Per ogni iterazione vengono salvati:

- la **ricostruzione** ottenuta dal NNF corrente
- la **visualizzazione del NNF**
- la **RMS map**

Inoltre viene tracciata la curva della RMS nel tempo e vengono mostrati separatamente gli input toy A e toy B.



Risultati osservati

- iter 0: RMS $\approx$ 94.94
- iter 1: RMS $\approx$ 18.22
- iter 2: RMS $\approx$ 11.90
- iter 3: RMS $\approx$ 11.12
- iter 4: RMS $\approx$ 10.82
- iter 5: RMS $\approx$ 10.58

Anche in questo caso si osserva una forte riduzione dell'errore nelle prime iterazioni, ma soprattutto il toy example rende **visivamente interpretabile** il metodo: da una inizializzazione casuale e incoerente si passa progressivamente a un NNF più regolare, a una ricostruzione più simile alla struttura corretta e a una mappa dell'errore più concentrata.

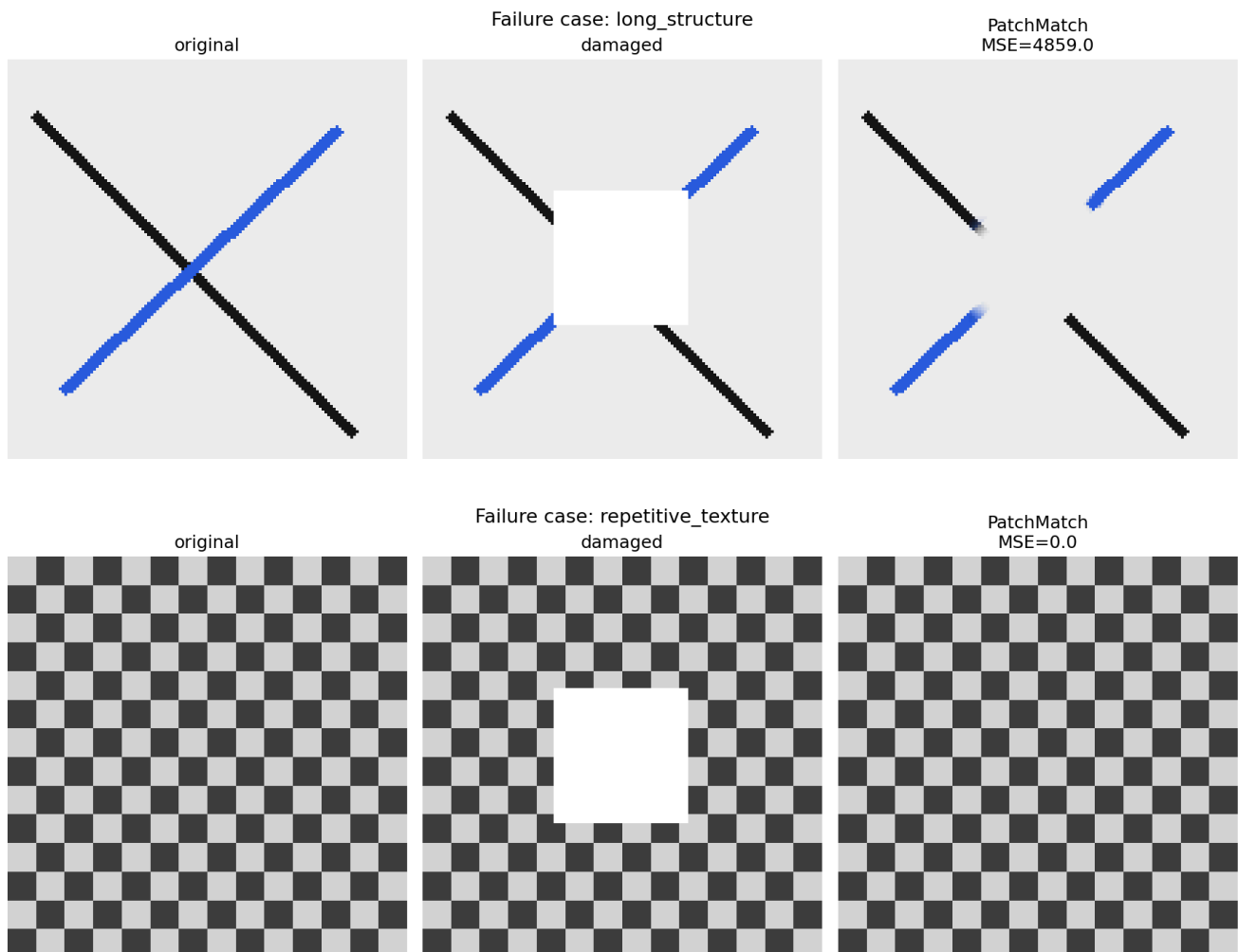
Questo esperimento è utile perché mostra non solo il risultato finale, ma i **passaggi intermedi** del metodo, cioè il modo in cui le corrispondenze migliorano iterazione dopo iterazione.

Esperimento 7 - Failure cases

L'esperimento raccoglie due casi in cui un approccio patch-based come PatchMatch-inpainting può andare in difficoltà:

- **texture ripetitive**
- **strutture lunghe che attraversano il buco**

Nel primo caso viene usata una texture a scacchiera; nel secondo, una scena sintetica con linee lunghe diagonali che devono essere ricostruite attraverso una regione mancante.



Risultati osservati

Nel caso di **texture ripetitiva**, il matching locale trova molte corrispondenze quasi equivalenti: il risultato può quindi apparire plausibile a livello locale, ma non necessariamente ricostruire in modo univoco la configurazione originaria.

Nel caso di **strutture lunghe**, il limite è più evidente: PatchMatch lavora bene quando può copiare e riaggregare pattern locali già presenti, ma fatica di più quando serve mantenere una **coerenza geometrica globale** attraverso il buco. In questi casi la continuità delle linee può degradarsi o risultare spezzata.

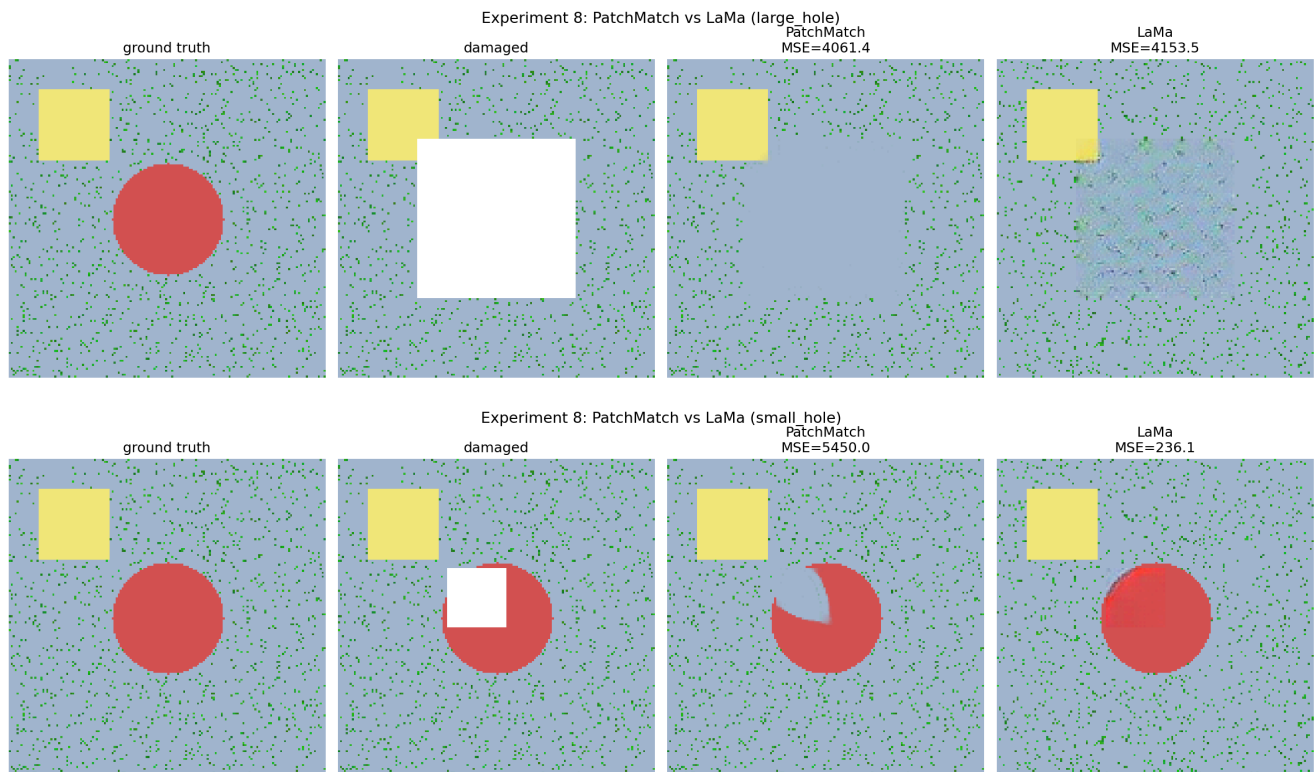
Questo esperimento è importante perché mostra che il metodo non va valutato solo sui casi in cui funziona bene, ma anche sui casi in cui la natura locale e patch-based del matching diventa un limite.

Esperimento 8 - Confronto PatchMatch vs LaMa

L'esperimento confronta l'inpainting PatchMatch con **LaMa**, usando un esempio sintetico realistico con due tipi di maschera:

- **small hole**
- **large hole**

Nel progetto, l'output di LaMa viene caricato da file precomputati nella cartella `lama_outputs`, mentre il risultato PatchMatch viene prodotto direttamente dal codice locale. Il confronto viene effettuato sia **visivamente** sia tramite **MSE sulla regione mascherata**.



Risultati osservati

Il confronto mette in evidenza una differenza di principio tra i due approcci:

- **PatchMatch** ricostruisce il contenuto copiando e riaggregando patch esistenti
- **LaMa** sfrutta un modello deep in grado di inferire strutture più globali e semanticamente coerenti

Nel caso di **buchi piccoli**, PatchMatch può ancora produrre risultati plausibili quando il contenuto da ricostruire è localmente ben supportato dal contesto.

Nel caso di **buchi grandi**, i limiti del metodo patch-based emergono più chiaramente: mantenere forma e coerenza globale diventa più difficile, mentre il metodo deep tende a ricostruire meglio strutture e contorni.

Questo esperimento non sostituisce un benchmark completo, ma è utile per collocare PatchMatch rispetto a metodi più moderni: il primo è molto interpretabile e direttamente legato al matching tra patch, mentre il secondo introduce una capacità di completamento più semantica.

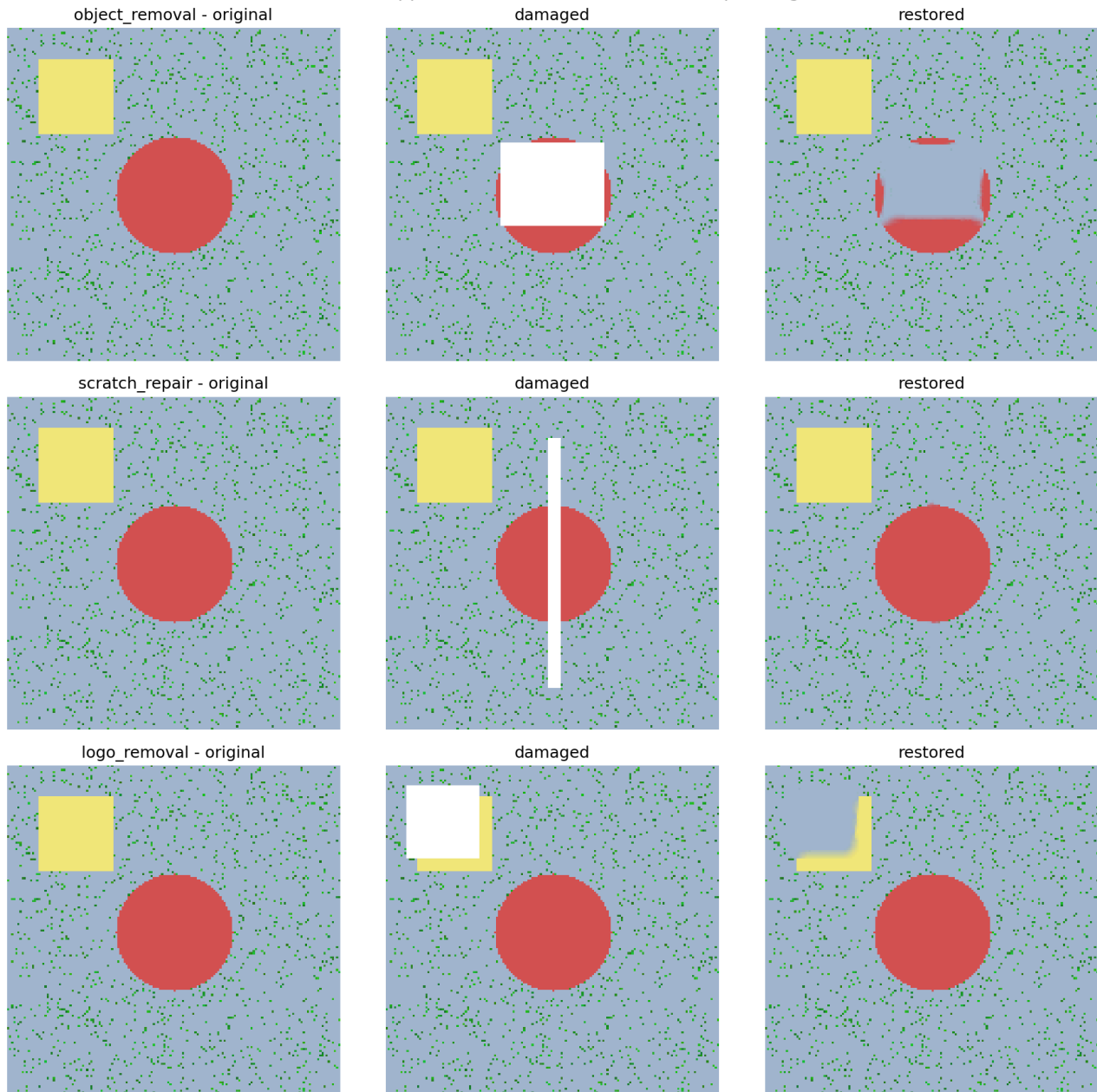
Esperimento 9 - Possibili applicazioni

L'ultimo esperimento costruisce un pannello qualitativo di possibili applicazioni di un approccio PatchMatch-based all'inpainting:

- **object removal**
- **scratch repair**
- **logo removal**

In tutti i casi si parte da una scena sintetica e si introduce una maschera specifica per simulare il tipo di danno o di editing desiderato.

Possible applications of PatchMatch-based inpainting



Risultati osservati

L'esperimento mostra che l'inpainting è una delle applicazioni più naturali di PatchMatch: quando l'informazione mancante può essere ricostruita riutilizzando porzioni già presenti dell'immagine, il meccanismo di search + vote risulta particolarmente adatto.

In particolare:

- nella **rimozione di piccoli oggetti** il metodo può funzionare bene se il contesto vicino contiene texture o pattern compatibili
- nel **ripristino di graffi o regioni sottili** l'approccio patch-based è naturale perché il problema è fortemente locale

- nella **rimozione di loghi o elementi sovrapposti** il risultato dipende molto dalla disponibilità, attorno alla maschera, di contenuto simile da cui copiare

Conclusioni sul progetto

Il progetto realizza una implementazione funzionale e sperimentabile di PatchMatch, separando:

- **core algoritmico** (`patchmatch.py`): NNF/NND, propagazione, random search, ricostruzione, storico dell'evoluzione e visualizzazione
- **validazione sperimentale** (`demo.py`): set di esperimenti riproducibili con metriche e figure
- **estensione applicativa**: inpainting multi-scala, failure cases, confronto con un metodo deep e pannello di applicazioni

Dal punto di vista sperimentale emergono i seguenti aspetti principali:

1. PatchMatch riduce drasticamente l'errore già nelle prime iterazioni
2. l'approssimazione rispetto all'ottimo brute force rimane contenuta
3. il toy example rende visivamente chiari i passaggi del metodo
4. il framework Search+Vote multi-scala rende naturale estendere il metodo a compiti di inpainting
5. i failure cases mostrano i limiti del matching puramente locale
6. il confronto con LaMa evidenzia la differenza tra approcci patch-based classici e metodi deep più recenti

Nel complesso, il progetto mostra sia i **punti di forza** di PatchMatch, cioè semplicità concettuale, efficienza pratica, interpretabilità del matching, sia alcuni suoi **limiti strutturali**, soprattutto quando serve ricostruire contenuti con forte coerenza globale o semantica.

Paper Visione Deep: Deformable Convolutional Networks

Introduzione

Le **Deformable Convolutional Networks (DCN)** introducono due moduli “drop-in” per reti CNN: la **deformable convolution** e la **deformable RoI pooling** (inclusa la variante **deformable position-sensitive RoI pooling**).

L'idea centrale è rendere **learnable** e quindi **content-adaptive** le posizioni di campionamento usate da convoluzioni e pooling su RoI: invece di usare una griglia regolare fissa, ogni punto del kernel o ogni bin della RoI viene spostato da un **offset** predetto dalla rete stessa. In questo modo il receptive field diventa più adattivo rispetto a forma, scala e pose degli oggetti, migliorando detection e segmentation con overhead relativamente contenuto.

Le CNN classiche hanno infatti un'assunzione geometrica implicita: i campioni che un filtro prende dall'input, o che un pooling prende da una regione, sono fissati su una **griglia regolare**. Questo è molto efficiente, ma può diventare un limite quando la geometria dell'oggetto è variabile:

- oggetti in pose diverse
- variazioni di scala
- contorni sottili o allungati
- deformazioni locali di parti

In una rete standard, due unità dello stesso layer campionano con la stessa geometria anche quando osservano contenuti molto diversi. La rete può compensare parzialmente con data augmentation e

profondità, ma la geometria rimane rigida: l'invarianza deve essere assorbita dai pesi, non dalla struttura di campionamento.

DCN cambia questo schema: lascia ai pesi del filtro il compito di decidere **cosa** estrarre, ma permette alla rete di imparare anche **da dove** estrarre.

Cosa c'era prima

Prima di DCN, il problema della robustezza geometrica veniva affrontato in modi diversi, ma nessuno introduceva un meccanismo locale, denso e adattivo esattamente nel punto in cui avviene il campionamento convoluzionale.

Una prima strategia è la **data augmentation**: mostrare alla rete molte varianti geometriche degli input. Questo migliora la robustezza statistica, ma non introduce un meccanismo esplicito di allineamento e richiede più dati e più training.

Le **dilated (atrous) convolution** aumentano il receptive field senza aumentare il numero di parametri, campionando su una griglia più ampia. Sono molto utili nella segmentazione, ma la geometria del campionamento rimane comunque **globale e fissa**: la stessa dilation viene usata ovunque, indipendentemente dal contenuto.

Gli **Spatial Transformer Networks (STN)** introducono invece un modulo differenziabile che stima una trasformazione e warpa input o feature map. È un'idea importante, ma in genere lavora con trasformazioni parametriche relativamente semplici, spesso globali o a bassa complessità, e non sostituisce il campionamento locale punto per punto di ogni kernel.

I **Deformable Part Models (DPM)** rappresentano gli oggetti come insiemi di parti con penalità di deformazione. Concettualmente sono vicini all'idea di spostare parti di un oggetto per migliorarne l'allineamento, ma non sono integrati come moduli deep generici e differenziabili su feature map moderne.

Infine, architetture come **R-FCN** costituiscono uno dei contesti sperimentali in cui gli autori valutano i moduli deformabili. In questo quadro, DCN si inserisce come estensione naturale di detector già esistenti, introducendo adattività geometrica locale, densa e addestrabile end-to-end, senza modificare la struttura di input e output dei moduli sostituiti.

Convoluzioni

Per semplicità si considerano feature map 2D, omettendo i canali, dato che la geometria di campionamento è condivisa sui canali.

Convoluzione standard

Sia R il set degli offset della griglia del kernel. Per esempio, in un kernel 3×3 si hanno $N = 9$ posizioni.

La convoluzione standard può essere scritta come:

$$y(p_0) = \sum_{p_n \in R} w(p_n) \cdot x(p_0 + p_n)$$

Qui:

- p_0 è la posizione corrente sulla feature map
- p_n sono i punti della griglia del kernel
- $w(p_n)$ sono i pesi del filtro
- $x(\cdot)$ è la feature map di input

La geometria del filtro è quindi identica ovunque.

Deformable convolution

L'idea è aggiungere, per ogni punto del kernel p_n , un offset 2D Δp_n dipendente dal contenuto locale.

La deformable convolution diventa:

$$y(p_0) = \sum_{p_n \in R} w(p_n) \cdot x(p_0 + p_n + \Delta p_n)$$

Interpretazione

- $w(p_n)$ decide **quanto** pesare il campione
- Δp_n decide **dove** prenderlo

Se $\Delta p_n = 0$ per tutti i punti, il modulo collassa esattamente nella convoluzione standard.

La differenza concettuale è che non si impone una trasformazione geometrica parametrica predefinita, ma si lascia che sia l'ottimizzazione a far emergere il pattern di campionamento più utile al task.

Interpolazione bilineare

Gli offset non sono vincolati a essere interi; quindi le posizioni $p_0 + p_n + \Delta p_n$ possono essere frazionarie. Per definire il valore della feature map in coordinate non intere, si usa l'interpolazione bilineare:

$$x(p) = \sum_q G(q, p) \cdot x(q)$$

dove q sono coordinate intere e G è il kernel bilineare:

$$G(q, p) = g(q_x, p_x) \cdot g(q_y, p_y), \quad g(a, b) = \max(0, 1 - |a - b|)$$

Questo è importante perché:

- rende il sampling differenziabile rispetto agli offset
- consente il passaggio dei gradienti durante il training
- in pratica coinvolge solo i 4 vicini della coordinata frazionaria

Come si predicono gli offset

La rete non riceve offset supervisionati: li apprende perché riducono la loss del task.

Gli offset sono prodotti da un layer aggiuntivo:

- si applica una **convoluzione** alla feature map di input
- l'output ha $2N$ canali, dove N è il numero di punti del kernel
- per ciascun punto del kernel si predicono due componenti: $(\Delta x, \Delta y)$

Deformable RoI pooling

RoI pooling standard

Dato un rettangolo RoI su una feature map, lo si divide in $k \times k$ bin e si esegue pooling all'interno di ciascun bin. Una forma average può essere scritta come:

$$y(i, j) = \frac{1}{n_{ij}} \sum_{p \in \text{bin}(i, j)} x(p_0 + p)$$

dove:

- (i, j) è l'indice del bin
- p_0 è l'origine della RoI
- n_{ij} è il numero di campioni nel bin

Deformable RoI pooling

DCN aggiunge un offset per ogni bin:

$$y(i, j) = \frac{1}{n_{ij}} \sum_{p \in \text{bin}(i, j)} x(p_0 + p + \Delta p_{ij})$$

In questo modo ogni bin può spostarsi verso la regione più informativa della RoI, migliorando l'allineamento tra la griglia rettangolare e la forma reale dell'oggetto.

Nel paper, gli offset del deformable RoI pooling non vengono ottenuti con lo stesso schema locale usato per la convoluzione: si applica prima un **RoI pooling standard** per ottenere una rappresentazione pooled della regione e, successivamente, un layer **fully connected** produce offset normalizzati $\Delta \hat{p}_{ij}$ per i vari bin. Anche in questo caso il campionamento può ricadere su coordinate frazionarie e viene quindi implementato tramite interpolazione bilineare.

Offset normalizzati

Gli offset vengono predetti in forma normalizzata e poi scalati rispetto alla dimensione della RoI:

$$\Delta p_{ij} = \gamma \cdot \Delta \hat{p}_{ij} \circ (w, h)$$

dove:

- (w, h) sono larghezza e altezza della RoI
- $\circ$ indica il prodotto di Hdamard
- γ è un fattore di scala

Questo permette di ottenere spostamenti proporzionali alla dimensione dell'oggetto. Nel paper, γ viene fissato a **0.1**, così da rendere l'apprendimento più stabile e coerente tra RoI di scala diversa.

Deformable position-sensitive RoI pooling

Nel caso di **R-FCN**, che usa mappe position-sensitive, DCN introduce una versione deformabile coerente con il paradigma fully convolutional:

- una convoluzione produce campi di offset
- un PS RoI pooling aggrega tali offset per la RoI
- gli offset spostano i campioni di ciascun bin

Più precisamente, ogni bin (i, j) non legge da una feature map generica, ma da una mappa posizione-specifica associata a quella cella della griglia. La versione deformabile mantiene questa struttura, ma aggiunge la possibilità di spostare localmente ciascun bin verso la parte più informativa dell'oggetto.

L'effetto pratico è un migliore allineamento delle parti dell'oggetto all'interno della RoI, mantenendo la struttura **position-sensitive** del framework **R-FCN**.

Integrazione nelle architetture complete

Uno dei punti di forza progettuali del paper è che i moduli proposti mantengono lo stesso tipo di input e output delle loro controparti standard. Possono quindi essere utilizzati come veri e propri **drop-in replacement** all'interno di architetture esistenti.

Gli autori li impiegano soprattutto con backbone come **ResNet-101** e **Aligned-Inception-ResNet**, costruendo su tali backbone modelli per:

- **DeepLab** nella semantic segmentation
- **Category-aware RPN**
- **Faster R-CNN**
- **R-FCN** nell'object detection

In generale:

- la **deformable convolution** viene inserita negli ultimi layer convoluzionali
- il **deformable RoI pooling** viene usato nella parte region-based dei detector

Gli esperimenti considerano l'inserimento dei moduli deformabili negli ultimi layer convoluzionali della backbone. I risultati mostrano che questa scelta è efficace e che **tre layer deformabili** costituiscono un buon compromesso tra complessità e prestazioni nelle configurazioni adottate dagli autori.

Receptive field adattivo ed effective dilation

Una parte importante del paper riguarda l'interpretazione qualitativa del comportamento della deformable convolution. Le figure mostrano che:

- nella convoluzione standard i punti campionati restano rigidi e identici in ogni posizione
- nella deformable convolution i punti si spostano e il receptive field si adatta all'oggetto osservato

Quando più layer deformabili vengono impilati, l'effetto si amplifica: il receptive field non ha più forma e dimensione fisse, ma varia in funzione del contenuto della scena. In particolare, sugli **oggetti grandi** tende ad allargarsi, mentre sugli **oggetti piccoli** tende a restare più concentrato; nelle regioni di **sfondo** assume valori medi compresi tra quelli osservati per oggetti medi e grandi.

Per quantificare questo comportamento, gli autori introducono anche la nozione di **effective dilation**, cioè una misura media della distanza tra punti adiacenti campionati da un filtro deformabile. Tale grandezza cresce al crescere della scala dell'oggetto e fornisce una conferma quantitativa del fatto che il modulo non si limita ad ampliare rigidamente il receptive field, ma ne adatta dinamicamente la geometria.

Confronto con la atrous convolution

Uno dei confronti più importanti proposti nel paper è quello con la **atrous convolution** o **dilated convolution**.

La dilated convolution aumenta il receptive field inserendo una distanza regolare tra i punti del kernel, ma lo fa in modo **fisso**. Gli autori sostengono che la **deformable convolution** costituisca una generalizzazione della **atrous convolution**, poiché può riprodurre campionamenti equivalenti a dilatazioni diverse e modellare pattern di campionamento adattivi dipendenti dal contenuto, inclusi casi coerenti con variazioni di **scala**, **aspect ratio anisotropo** e **rotazione**.

I risultati sperimentali mostrano che:

- aumentare la dilatazione rispetto al baseline migliora effettivamente le prestazioni in diversi task
- tuttavia, il valore ottimale di dilatazione fissa varia da problema a problema

- la deformable convolution ottiene in generale risultati migliori rispetto alle alternative con dilatazione costante

Quindi non è sufficiente avere un receptive field più ampio; è decisivo poter disporre di un receptive field **adattivo**.

Effetto del deformable RoI pooling e combinazione dei moduli

Il paper mostra che anche l'impiego del solo **deformable RoI pooling** migliora sensibilmente le prestazioni, in particolare nelle metriche di localizzazione più severe.

Questo risultato conferma l'intuizione di fondo, cioè che spostare i bin della RoI consente una migliore corrispondenza tra la griglia di pooling e la struttura effettiva dell'oggetto, con effetti positivi soprattutto sulla precisione di localizzazione.

Quando **deformable convolution** e **deformable RoI pooling** vengono usati nella stessa architettura, i risultati migliorano ulteriormente rispetto all'uso di ciascun modulo da solo. I dati sperimentali indicano quindi che l'uso congiunto dei due moduli è particolarmente efficace, combinando un'estrazione di feature più adattiva con un migliore allineamento delle regioni di interesse.

Numero di layer deformabili

Il paper analizza anche quanti layer deformabili sia opportuno introdurre nelle backbone convoluzionali.

I risultati mostrano che:

- l'inserimento di deformable convolution negli ultimi layer migliora sistematicamente le prestazioni rispetto al baseline
- l'aumento del numero di layer deformabili porta a miglioramenti progressivi
- oltre una certa soglia si osserva una **saturazione** del beneficio

Gli autori scelgono come configurazione di default **tre layer deformabili** negli strati alti del backbone. Questa scelta è coerente con l'idea che la deformabilità sia particolarmente utile nelle fasi più profonde, dove la rete deve rappresentare strutture più complesse e semanticamente ricche.

Complessità e runtime

Un aspetto rilevante è comprendere se i miglioramenti ottenuti dipendano semplicemente da un aumento del numero di parametri.

Il paper mostra che l'overhead introdotto dai moduli deformabili è **relativamente contenuto**:

- il numero di parametri aumenta solo moderatamente
- anche i tempi di forward e di runtime crescono, ma in misura limitata rispetto ai guadagni prestazionali

Gli autori concludono quindi che il vantaggio del metodo non deriva semplicemente da una maggiore capacità parametrica, ma soprattutto da una **modellazione più efficace delle trasformazioni geometriche**.

Questo punto rafforza il valore scientifico della proposta: l'incremento di accuratezza non è spiegabile solo con il fatto di aver reso la rete più grande, ma con l'introduzione di un meccanismo strutturalmente più adatto al problema.

Risultati sperimentali su COCO

Il paper valida inoltre il metodo sul benchmark **COCO test-dev**, oltre che su **PASCAL VOC**. I risultati mostrano miglioramenti consistenti per diverse architetture, sia con backbone **ResNet-101** sia con backbone **Aligned-Inception-ResNet**.

I risultati mostrano miglioramenti consistenti per diverse architetture, sia con backbone **ResNet-101** sia con backbone più forti come **Aligned-Inception-ResNet**. Il vantaggio delle deformable ConvNets si mantiene anche quando le baseline vengono rafforzate, segnalando che il contributo del metodo è **complementare** rispetto ad altri miglioramenti architetturali o tecnici.

Inoltre, l'uso di accorgimenti aggiuntivi come:

- **multi-scale testing**
- **iterative bounding box average**

porta ulteriori miglioramenti, senza annullare il contributo specifico dei moduli deformabili. Questo indica che la proposta si integra bene con altre tecniche avanzate di object detection.

Confronto con i lavori correlati

Il paper confronta il proprio approccio con diversi filoni precedenti.

Spatial Transformer Networks

Gli **Spatial Transformer Networks (STN)** apprendono trasformazioni spaziali dai dati, ma tipicamente utilizzano una trasformazione parametrica globale, come una trasformazione affine. La proposta di DCN, invece, opera in modo:

- **locale**
- **denso**
- **adattivo**
- senza imporre una trasformazione globale unica

Per questo risulta più naturale in task di previsione densa o semi-densa come detection e segmentation.

Active Convolution

Rispetto ad approcci come **Active Convolution**, la differenza principale è che in quei metodi gli offset sono generalmente **statici e condivisi nello spazio**, mentre in DCN gli offset sono **dinamici**, dipendenti sia dall'immagine sia dalla posizione spaziale.

Deformable Part Models

Il collegamento con i **Deformable Part Models (DPM)** riguarda l'idea di modellare la deformazione delle parti dell'oggetto. Tuttavia, DCN integra questa idea all'interno di una rete profonda:

- addestrabile end-to-end
- priva delle euristiche tipiche dei DPM classici
- applicata direttamente a feature map profonde

Spatial Pyramid Pooling e pooling progettato manualmente

Rispetto a tecniche come **spatial pyramid pooling** o altre forme di pooling progettate manualmente, il paper sottolinea di introdurre un meccanismo semplice e diretto per apprendere end-to-end **regioni di pooling deformabili** all'interno delle CNN.

Conclusione

Il contributo fondamentale di DCN è mostrare che la rigidità geometrica delle CNN può essere attenuata direttamente nel meccanismo di campionamento, senza sostituire l'intera architettura e senza richiedere supervisione esplicita sugli offset.

Il punto chiave non è soltanto l'aumento di flessibilità, ma il fatto che tale flessibilità venga introdotta nel posto giusto: nel modo in cui il filtro legge le feature map e nel modo in cui la RoI viene allineata all'oggetto.

I risultati sperimentali del paper mostrano che questa scelta produce miglioramenti consistenti in **object detection** e **semantic segmentation**, superando non solo le versioni standard dei moduli, ma anche alternative con receptive field più ampio ma fisso, come le **dilated convolutions**. Il rapporto tra incremento di complessità e guadagno prestazionale risulta quindi favorevole.

Progetto su Deformable Convolutional Networks

Questo progetto implementa, in forma semplificata, il contributo centrale del paper, cioè l'idea di sostituire il campionamento a griglia fissa delle convoluzioni standard con un campionamento **deformabile** i cui offset sono **appresi** a partire dalle feature map.

L'obiettivo non è stato replicare l'intera pipeline della repository ufficiale, che è orientata al *detection* e quindi GPU dipendente, bensì:

- realizzare una versione **utilizzabile su CPU**
- testare la repo per fare degli esperimenti
- produrre output riproducibili: curve di training, metriche, grafici di robustezza e visualizzazioni degli offset

Il progetto è strutturato attorno a tre esperimenti:

1. confronto Standard CNN vs Deformable CNN su **MNIST** e **FashionMNIST**
2. visualizzazione qualitativa degli **offset appresi**
3. misura della **robustezza** a trasformazioni geometriche (rotazione, scala, shear)

Repository originale

repo: <https://github.com/msracver/Deformable-ConvNets.git>

La repository ufficiale del paper è progettata principalmente per compiti di **object detection** (e task affini) e contiene:

1. **Operatori custom (C++/CUDA)** per *deformable convolution* e *deformable RoI pooling*.
2. Backbone e **Pipeline di training/evaluation** per modelli di detection (es. varianti tipo Faster R-CNN / R-FCN con integrazione dei layer deformabili)
3. **Configurazioni e script** orientati a dataset complessi (COCO/PASCAL), con dipendenza da GPU per i costi computazionali

Questo rende la repo ottima come riferimento e baseline di implementazione, ma poco pratica su CPU.

Cosa è stato preso dalla repository

È stata mantenuta la struttura generale **conv standard + branch offsets**:

- un ramo convoluzionale aggiuntivo predice gli offset
- gli offset deformano il pattern di campionamento della convoluzione
- gli offset del ramo sono inizializzati a zero, così da far partire la rete come baseline standard

Per un kernel $k \times k$, con $N = k^2$ punti di campionamento, il ramo offset produce $2N = 2k^2$ **canali**.

È stato inoltre scelto di inserire la deformable convolution solo negli **ultimi layer**, mantenendo i primi layer standard. Questa scelta è coerente con l'idea che ai livelli più alti le feature siano più semantiche e possano beneficiare maggiormente di un receptive field adattivo.

Modifiche per implementazione CPU

- Pipeline in **PyTorch**
- Sostituzione dell'operatore custom CUDA della repository con `torchvision.ops.DeformConv2d`, eseguibile anche su CPU
- Sostituzione del contesto detection con un contesto di **classification** su MNIST/FashionMNIST, così da rendere:
 - il training più rapido e riproducibile
 - più immediate le visualizzazioni degli offset
 - più semplici gli esperimenti di robustezza, senza introdurre bounding box o RoI pooling

File del progetto

Il progetto è strutturato attorno a cinque esperimenti:

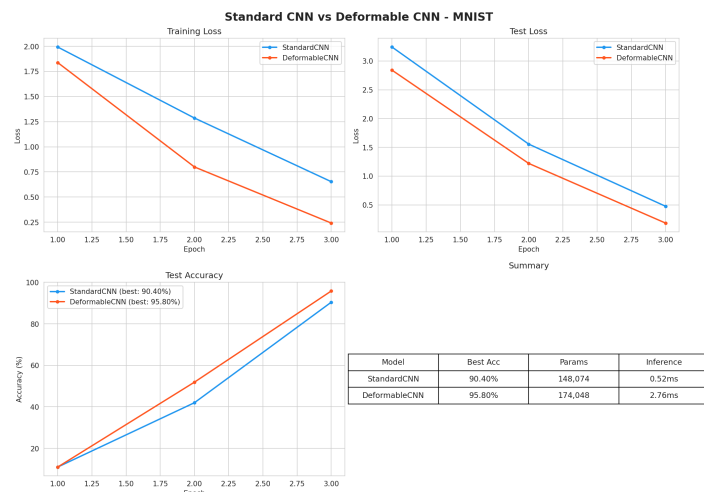
1. confronto **StandardCNN vs DeformableCNN** su **MNIST** e **FashionMNIST**
2. visualizzazione qualitativa degli **offset appresi**
3. misura della **robustezza** a trasformazioni geometriche (**rotazione, scala, shear**)
4. costruzione di **toy visualizations** per spiegare visivamente il funzionamento di convoluzione standard, convoluzione deformabile, pooling e campionamento con **interpolazione bilineare**
5. confronto qualitativo delle **feature responses** nei layer standard e deformabili, per osservare come cambia la risposta dei filtri nei diversi stadi della rete

I nuovi esperimenti 4 e 5 non modificano il modello né il training, ma estendono la parte di **interpretabilità** del progetto. L'idea è passare da una semplice osservazione degli offset a una lettura più completa del comportamento del meccanismo deformabile: da un lato con esempi didattici controllati, dall'altro con un confronto diretto tra le attivazioni interne delle reti standard e deformabile.

Valutazione risultati

Esperimento 1

Dataset MNIST



La DeformableCNN raggiunge una **test accuracy finale di 95.8%**, contro **90.4%** della StandardCNN.

La lettura più corretta di questo risultato è che, nel setup sperimentale considerato, la DeformableCNN mostra una migliore capacità di adattarsi alla geometria locale dei digit, ottenendo una performance di test superiore dopo lo stesso numero di epoche.

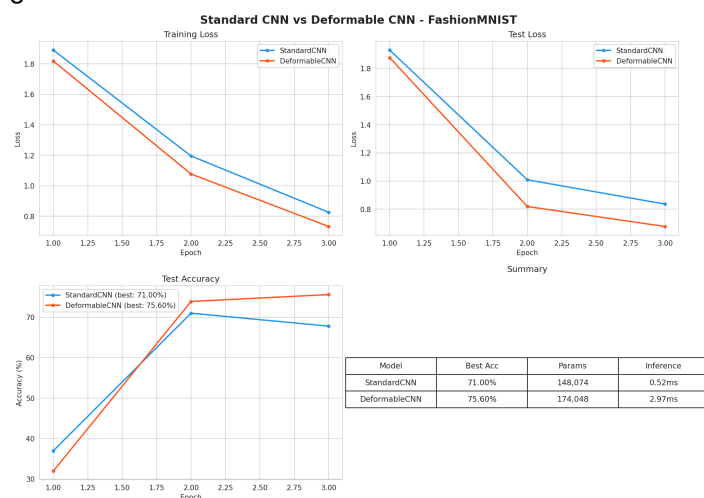
Costo

- Parametri: **148,074** → **174,048**
- Inference: **0.52 ms** vs **2.76 ms**
- Tempo per epoca: circa **3.6s** vs **106s**

Quindi il guadagno in accuracy è accompagnato da un overhead significativo, soprattutto su CPU.

Dataset FashionMNIST

FashionMNIST è più complesso (texture, forme più variabili), quindi è più realistico per la “capacità geometrica” del modello.



Anche su FashionMNIST la DeformableCNN mantiene **test loss più bassa** e **test accuracy più alta: 75.6%** contro **71.0%**.

La StandardCNN mostra un comportamento meno stabile nelle ultime epoche, mentre la DeformableCNN continua a migliorare fino alla terza epoca. Nel contesto di questo esperimento, è corretto parlare di **migliore performance di test nel setup considerato**, non di prova generale di migliore generalizzazione.

Costo

- Inference: **0.52 ms** vs **2.97 ms**
- Tempo per epoca deformabile: circa **106-109s**

Poiché le accuracy sono misurate su subset ridotti, i valori assoluti non sono confrontabili con benchmark SOTA, ma restano adeguati per un confronto controllato tra architetture.

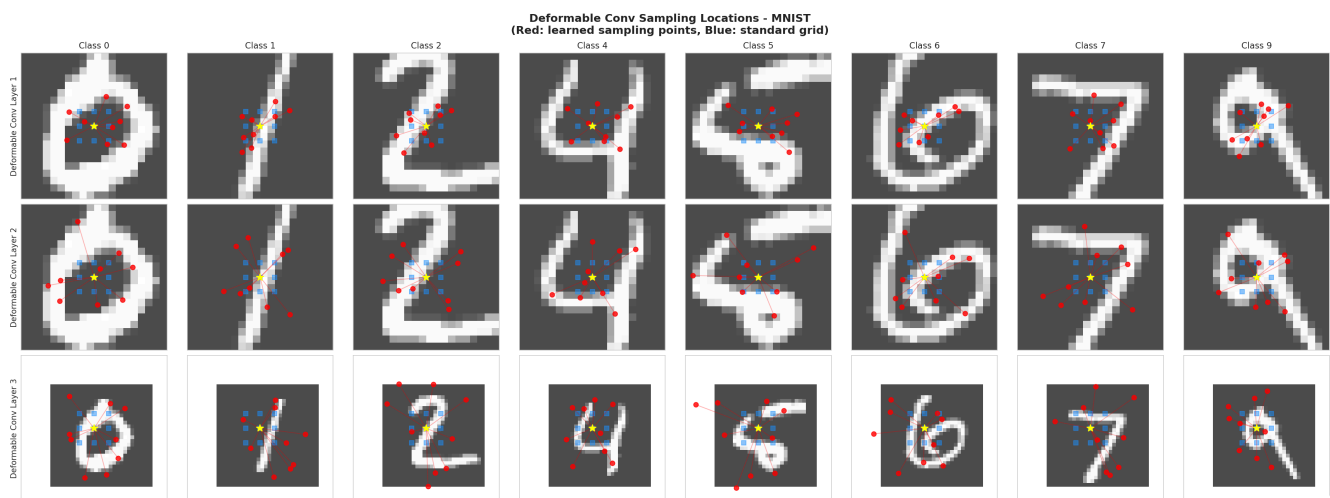
Esperimento 2

Questo esperimento serve a collegare il **meccanismo** alla **prestazione**, mostrando come il campionamento venga modificato dagli offset appresi.

Come leggere le figure

- **Blu** = griglia regolare 3×3
- **Rosso** = punti di campionamento dopo l'applicazione degli offset appresi
- **Stella gialla** = posizione centrale di riferimento

Dataset MNIST

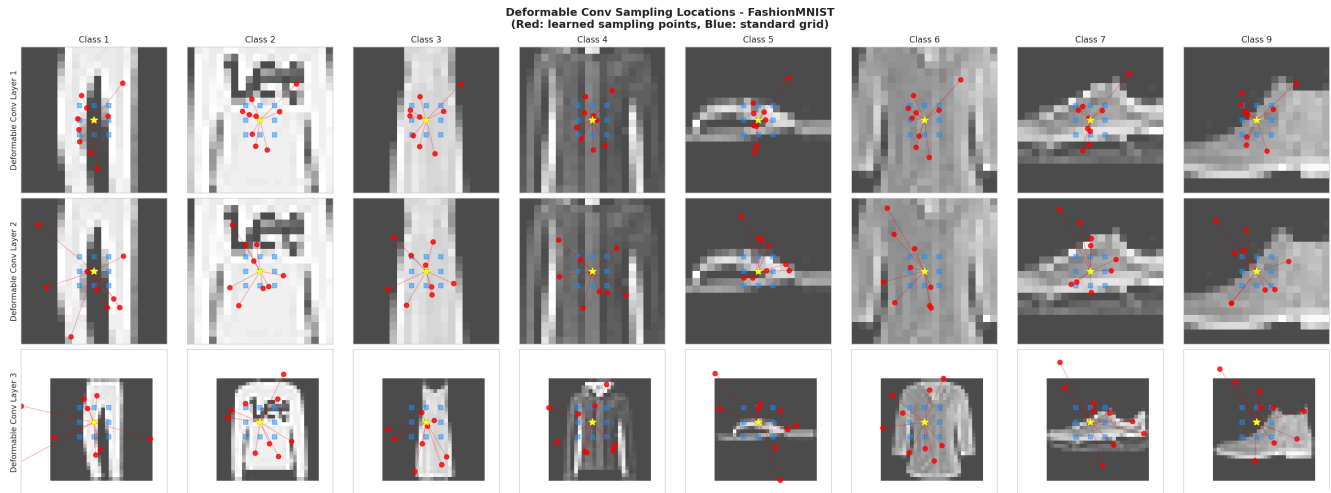


Nei digit sottili, come “1”, i punti rossi **sembrano** disporsi più spesso lungo la direzione dominante dello stroke. Questo suggerisce che il kernel non campiona più in modo perfettamente simmetrico, ma adatti il proprio supporto alle regioni localmente più informative.

Nei digit curvi, come “0”, “6” e “9”, i punti rossi **appaiono compatibili** con un campionamento che segue meglio la geometria locale della cifra.

Andando da **Deform Layer 1** a **Deform Layer 3**, l'ampiezza degli spostamenti tende ad aumentare, suggerendo qualitativamente che i layer più profondi si prendano maggiore libertà nel ridefinire il campo recettivo.

Dataset FashionMNIST



Su oggetti con struttura più complessa, i punti rossi **sembrano** spostarsi più spesso verso bordi e contorni, cioè regioni potenzialmente più informative dal punto di vista discriminativo.

Anche qui, nei layer più profondi la deformazione appare più marcata. Questa osservazione è coerente con l'idea che le deformable convolution possano adattarsi meglio a variazioni di forma intra-classe, ma la visualizzazione resta un'evidenza interpretativa, non una prova diretta.

Esperimento 3

Questo esperimento valuta la robustezza dei modelli rispetto a trasformazioni geometriche tramite test set trasformati con **rotazione**, **scala** e **shear**.

Transformed samples

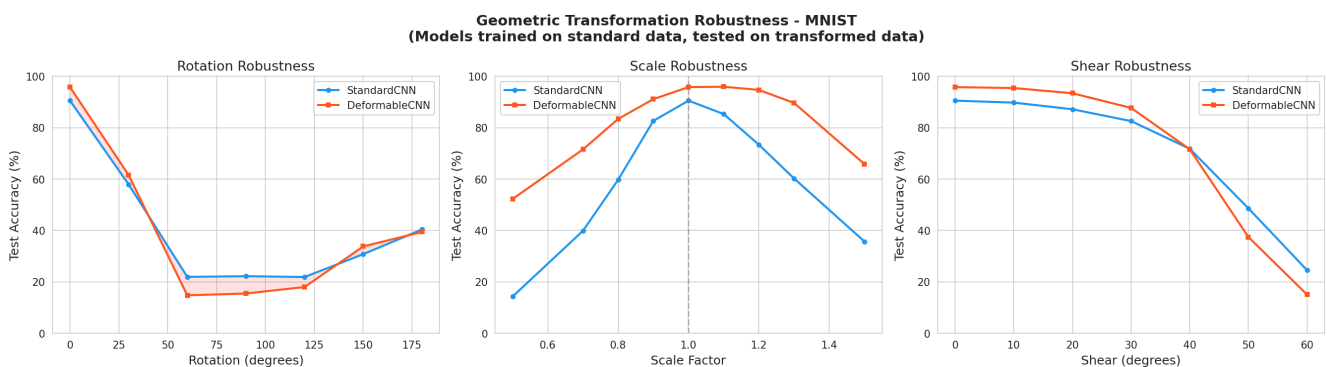


Le immagini trasformate mostrano che:

- le rotazioni forti introducono spesso cropping, aliasing e orientamenti molto distanti dal training set
- le scale piccole riducono drasticamente il numero di pixel informativi
- shear elevati deformano in modo pesante la geometria locale

Geometric robustness

Dataset MNIST



Rotazione

La DeformableCNN tende a mantenere un vantaggio per rotazioni moderate, ma per rotazioni molto forti entrambe le reti degradano pesantemente. Questo è coerente con il fatto che la deformabilità locale non equivale a una vera rotation equivariance.

Scala (0.5 → 1.5)

Qui emerge il risultato più forte: la DeformableCNN è **nettamente più robusta**.

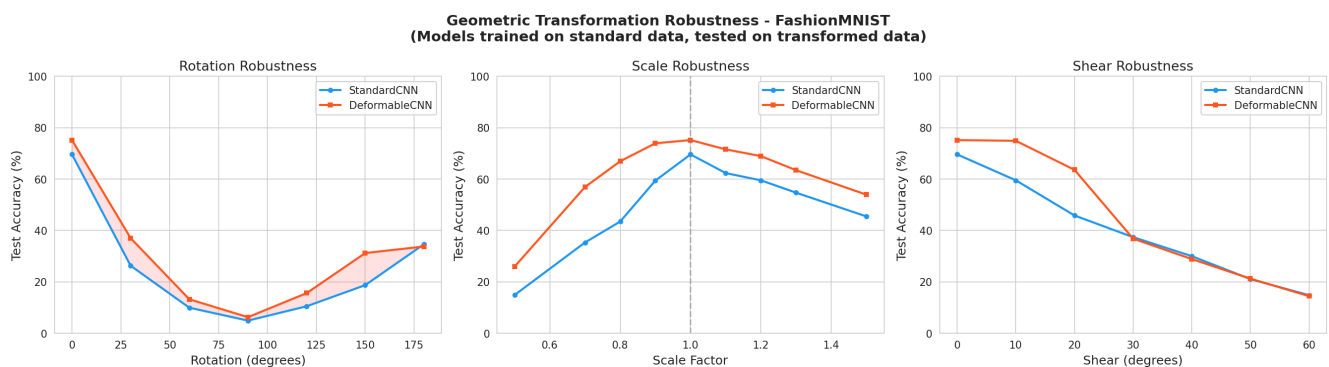
- Scale 0.5: **52.3% vs 14.3%** (gap enorme)
- Scale 1.2: **94.7% vs 73.4%**
- Scale 1.5: **65.9% vs 35.7%**

Con scala diversa, l'oggetto occupa porzioni diverse del campo recettivo. La deformable conv può **spostare i sampling points** per "rincorrere" il segnale e preservare la qualità delle feature anche quando la struttura appare più piccola o più grande.

Shear (0 → 60)

Per shear piccoli e medi la DeformableCNN mantiene un vantaggio, ma per shear elevati degrada più rapidamente e può incrociarsi con la StandardCNN. Questo suggerisce che la deformabilità locale aiuti fino a un certo punto, ma non sostituisca un training esplicito su trasformazioni molto estreme.

Dataset FashionMNIST



Rotazione

La DeformableCNN è quasi sempre superiore fino a rotazioni molto elevate:

- 30°: **37.0% vs 26.3%**
- 90°: **6.3% vs 4.9%**
- 150°: **31.2% vs 18.7%**

A 180° i risultati diventano molto simili. Anche qui, oltre una certa soglia la trasformazione altera troppo la distribuzione naturale del dataset.

Scala

La DeformableCNN mantiene un vantaggio consistente lungo tutto l'intervallo:

- Scale 0.7: **56.9% vs 35.3%**
- Scale 1.5: **53.9% vs 45.5%**

Rispetto a MNIST, il guadagno è meno drammatico ma più uniforme.

Shear

Il vantaggio è evidente per shear piccoli, poi le curve tendono a convergere e degradare insieme. Oltre una certa soglia, la sola deformabilità non basta più e diventa importante anche l'augmentation esplicita.

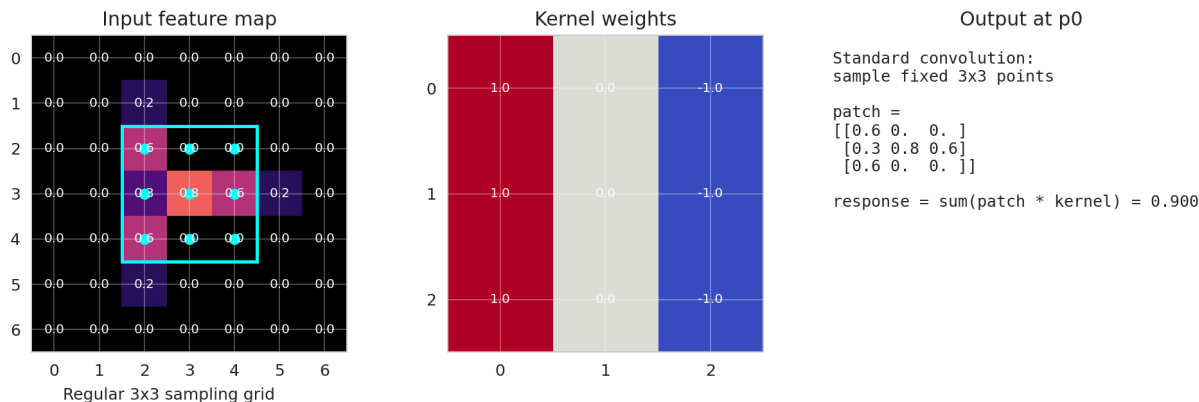
Esperimento 4

Questo esperimento è stato introdotto per soddisfare un obiettivo diverso dai primi tre: non misurare soltanto *quanto* la DeformableCNN performi meglio nel setup considerato, ma anche spiegare in modo visivo **che cosa fa** il meccanismo deformabile e come differisce da una pipeline standard.

L'idea è usare **toy examples** piccoli e controllati, nei quali la lettura della figura sia immediata. In questo modo il comportamento dei layer non resta implicito nella formula, ma diventa osservabile direttamente.

Toy visualizations

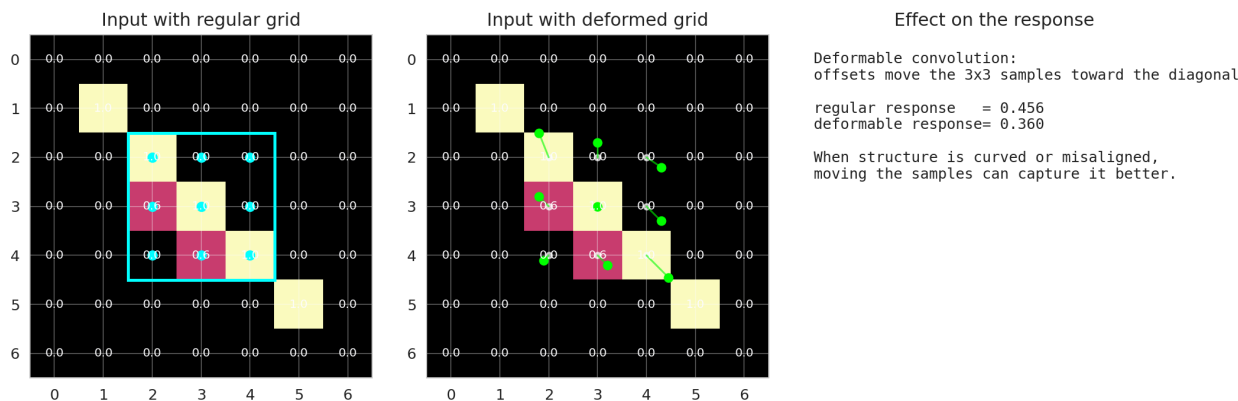
Convoluzione standard



Nel toy example della convoluzione standard si vede una griglia regolare **3×3** applicata a una piccola feature map. Il punto importante è che il pattern di campionamento è **fisso**: la convoluzione legge sempre gli stessi punti relativi rispetto al centro, indipendentemente dalla geometria locale del segnale.

Questa è la caratteristica fondamentale della convoluzione classica: la traslazione del kernel nello spazio è regolare, ma la forma del supporto di campionamento non cambia mai. Se una struttura è allineata con il filtro, la risposta può essere elevata; se invece la struttura è curvata, inclinata o leggermente spostata rispetto alla griglia, il kernel continua comunque a campionare gli stessi punti.

Convoluzione deformabile

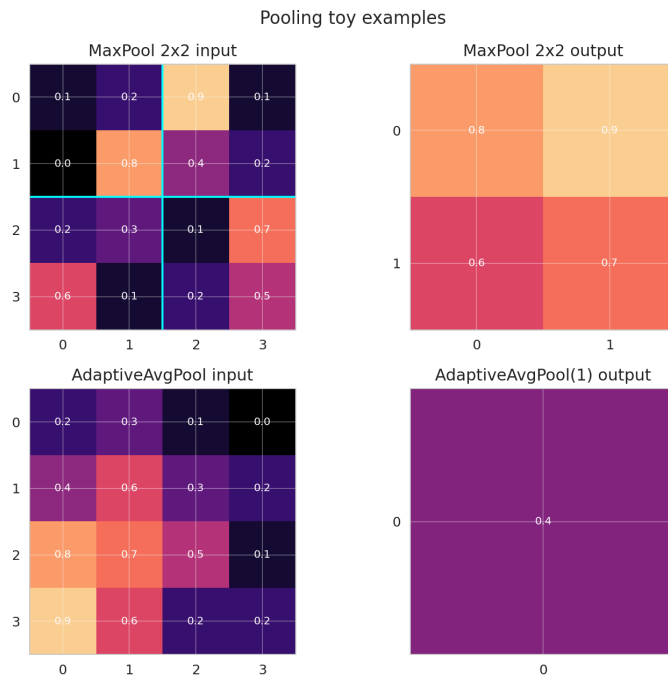


Nel toy example della convoluzione deformabile la griglia di campionamento non resta più regolare: i punti vengono spostati da **offset appresi**, e si vede visivamente che il pattern si piega verso la struttura rilevante della feature map.

Questo è il punto concettuale centrale del paper: la rete non modifica i pesi del filtro in funzione della posizione, ma modifica **dove** il filtro va a leggere l'informazione. In altre parole, la deformable convolution rende adattivo il **supporto spaziale** del kernel.

Nel toy example il messaggio non è che la risposta numerica debba essere sempre “più grande” in assoluto, ma che la risposta cambia perché il campionamento non è più vincolato alla griglia fissa. Quindi il meccanismo deformabile introduce un grado di libertà spaziale che la convoluzione standard non possiede.

Pooling

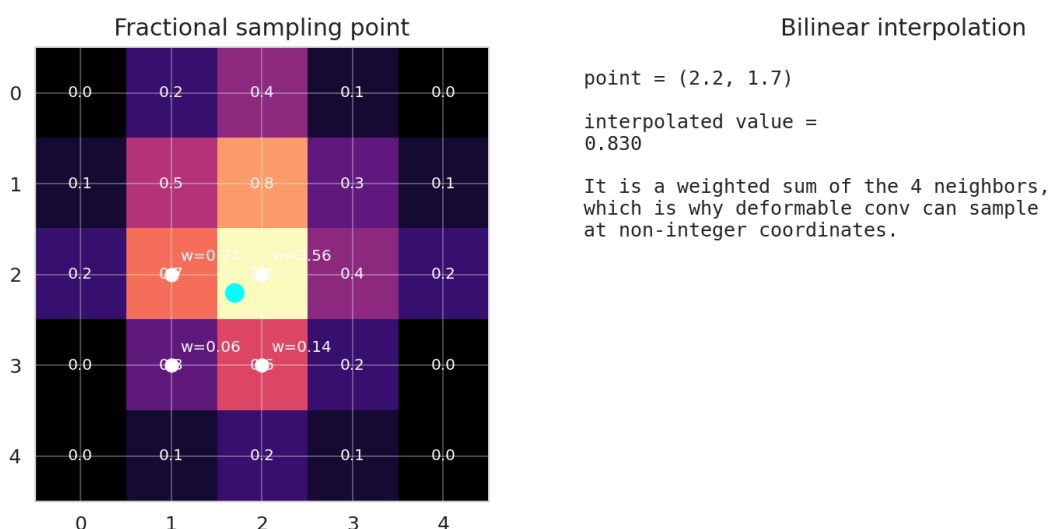


La visualizzazione del pooling chiarisce due aspetti diversi della pipeline.

Nel caso di **MaxPool 2×2**, la rete mantiene, in ciascuna finestra locale, il valore massimo. Questo equivale a una selezione competitiva dell’attivazione più forte e produce una riduzione di risoluzione mantenendo i segnali più salienti.

Nel caso di **AdaptiveAvgPool(1)**, invece, l’intera feature map viene compressa in un unico valore medio per canale. Questa operazione perde quasi completamente l’informazione spaziale fine, ma conserva una sintesi globale della presenza della feature. È proprio questa compressione finale che consente alla rete di passare da mappe spaziali a una rappresentazione adatta alla classificazione.

Interpolazione bilineare



La figura sull'interpolazione bilineare è particolarmente importante perché spiega un dettaglio tecnico spesso lasciato implicito: quando i punti di campionamento deformati cadono in coordinate **non intere**, il valore letto non coincide con un singolo pixel, ma con una combinazione pesata dei quattro vicini.

Nel toy example si vede un punto di campionamento frazionario e i relativi pesi assegnati ai pixel adiacenti. Questo rende chiaro perché la deformable convolution possa “spostarsi” nello spazio in modo continuo e non soltanto a salti discreti: l'offset appreso non seleziona un nuovo pixel intero, ma modifica il punto di lettura in modo continuo tramite interpolazione.

Interpretazione dell'esperimento 4

Questo esperimento rafforza la lettura degli esperimenti precedenti. Gli offset visualizzati nell'esperimento 2 non sono soltanto vettori astratti, ma la traduzione operativa di un cambio di campionamento; i risultati di robustezza dell'esperimento 3, in particolare su variazioni di scala, diventano più comprensibili proprio alla luce di questi toy examples.

In sintesi, le toy visualizations mostrano che:

- la **convoluzione standard** usa una griglia rigida
- la **convoluzione deformabile** sposta i punti di campionamento verso regioni più informative
- il **pooling** riduce la dimensionalità mantenendo informazione saliente o media globale
- l'**interpolazione bilineare** rende possibile il campionamento deformabile in coordinate non intere

Questa parte non dimostra da sola un vantaggio quantitativo, ma aggiunge una spiegazione visiva molto utile del perché il modello deformabile possa adattarsi meglio a strutture locali non perfettamente allineate.

Esperimento 5

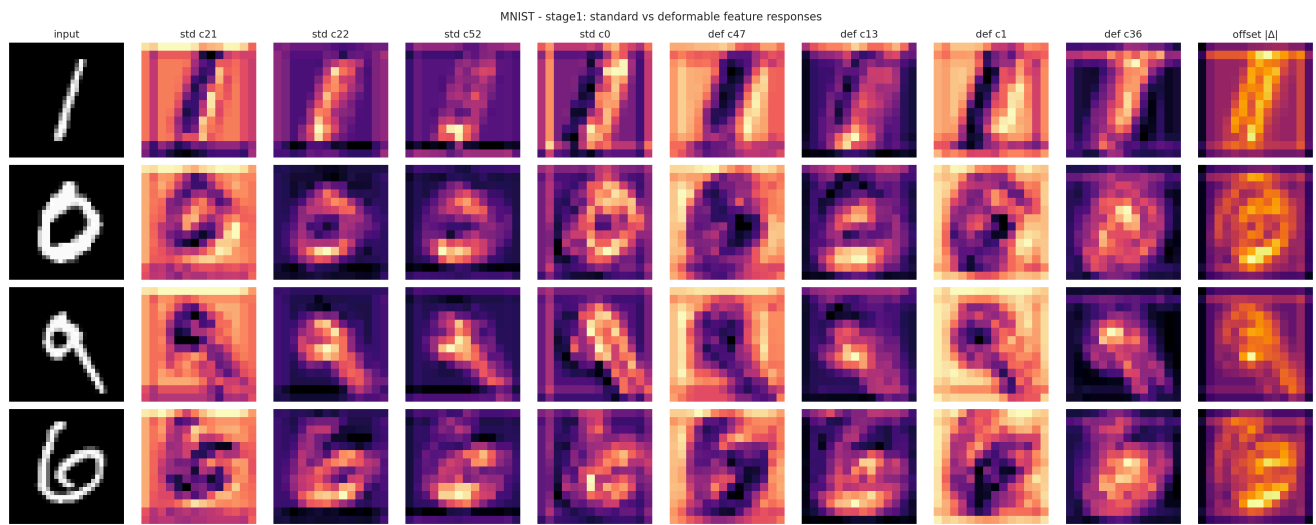
Dopo aver visualizzato gli offset appresi e aver introdotto toy examples dei layer, è utile osservare direttamente anche la risposta interna della rete. Per questo è stato aggiunto un quinto esperimento che confronta, in alcuni stadi selezionati, le **feature maps** generate dalla **StandardCNN** e dalla **DeformableCNN**.

Dal punto di vista metodologico, i canali mostrati nelle figure non sono scelti manualmente “a occhio”, ma selezionati automaticamente tra quelli con maggiore attivazione media assoluta. Questa scelta non garantisce che il filtro sia semanticamente interpretabile in senso forte, ma riduce il rischio di selezionare mappe poco informative e rende il confronto più sistematico.

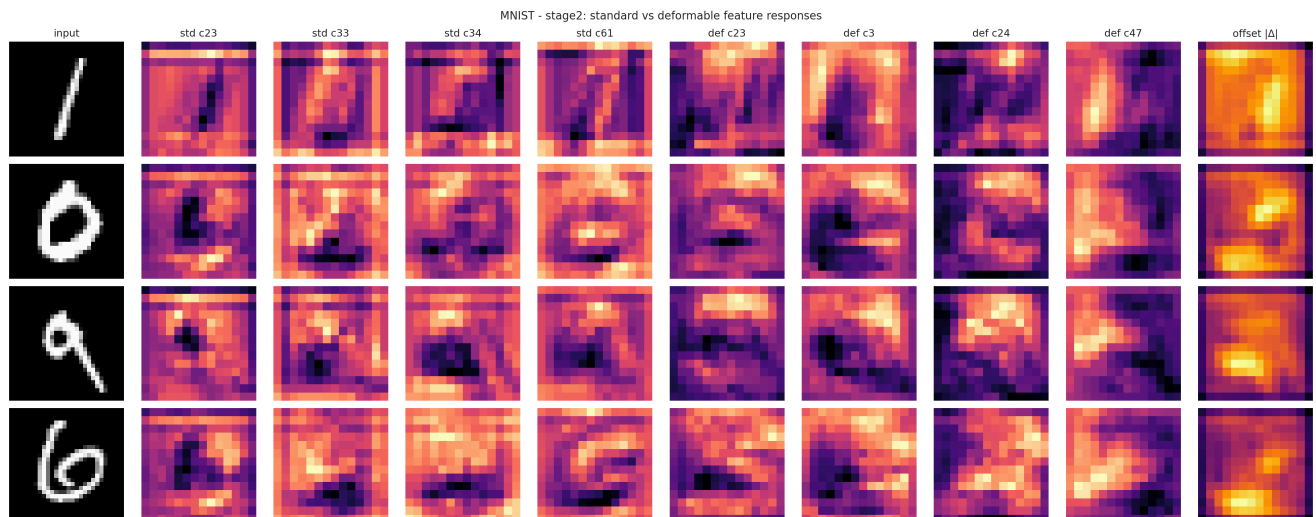
L'idea non è confrontare tutte le feature map possibili, ma selezionare alcuni **canali rilevanti** e metterli a confronto a parità di input. Nel codice, i canali mostrati vengono scelti automaticamente tra quelli con risposta media assoluta più elevata su un piccolo support set. Questo non fornisce un'interpretazione semantica completa del singolo filtro, ma permette di concentrare l'analisi su attivazioni effettivamente informative.

Per **MNIST** vengono mostrati esempi di classi come **1, 0, 9 e 6**, mentre per **FashionMNIST** si usano classi come **trouser, sandal, sneaker e bag**, cioè oggetti con geometrie e contorni significativamente diversi.

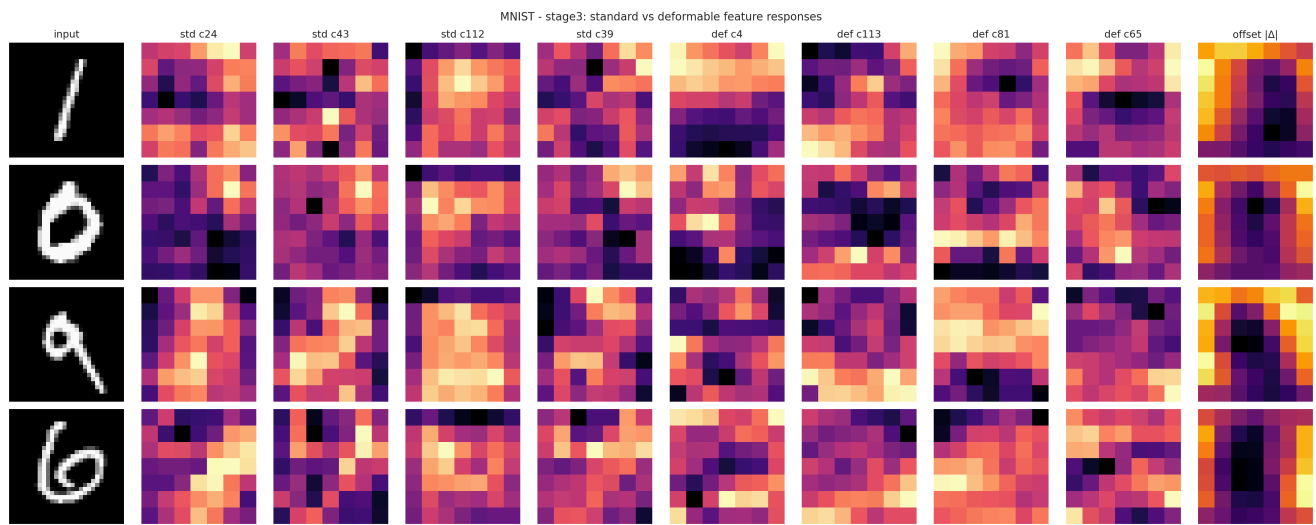
MNIST - stage 1, stage 2, stage 3



Nel **primo stadio** le differenze tra standard e deformabile sono già visibili, ma restano ancora fortemente legate alla geometria elementare dello stroke. Per il digit “1”, ad esempio, diverse feature deformabili appaiono più concentrate lungo la struttura verticale dominante; per digit come “0”, “6” e “9”, alcune mappe deformabili sembrano seguire in modo più aderente curve, anelli e regioni di cambiamento della forma. Le mappe standard, invece, risultano spesso più rigide o più diffuse rispetto alla struttura della cifra.



Nel **secondo stadio** il confronto diventa più interessante: le feature map sono meno “pixel-like” e più orientate a pattern di forma. In questa fase le attivazioni della rete deformabile appaiono spesso più selettive, con regioni luminose maggiormente concentrate sulle parti effettivamente descrittive della cifra. Parallelamente, le heatmap della magnitudine degli offset mostrano che l’attenzione deformabile si concentra nelle zone occupate dal digit e molto meno sullo sfondo, coerentemente con l’idea che il receptive field venga adattato dove il segnale è realmente presente.

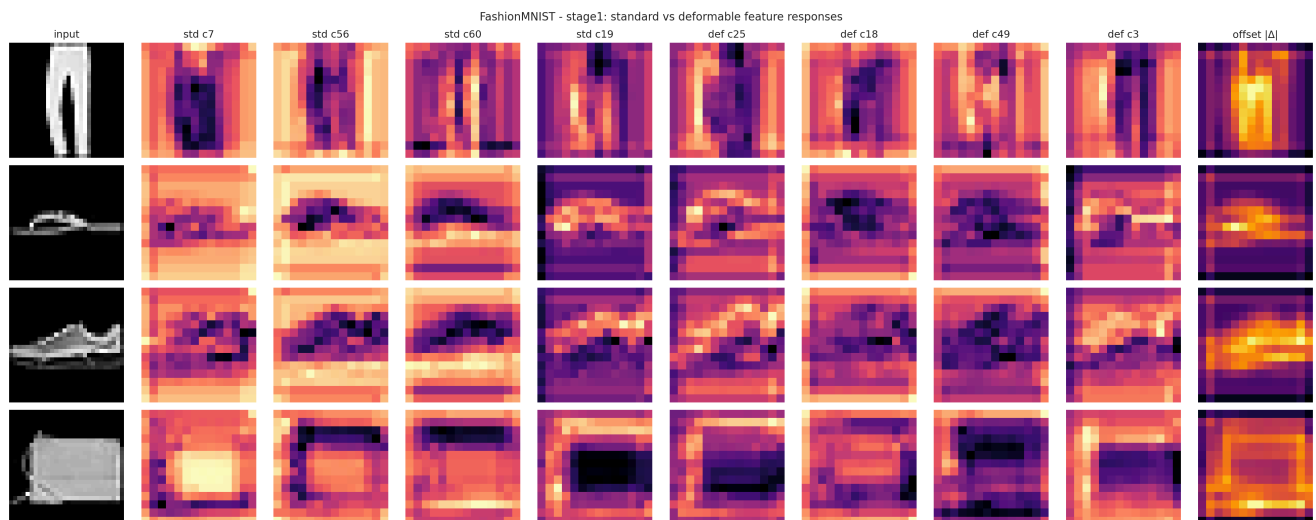


Nel **terzo stadio** la risoluzione spaziale è ormai molto ridotta e le mappe diventano inevitabilmente più astratte e difficili da interpretare in modo locale. Tuttavia, anche a questo livello si osserva che la rete deformabile continua a produrre attivazioni non uniformi e offset con distribuzione non banale. Questo suggerisce che la deformabilità non agisca solo come correzione superficiale dei primi layer, ma continui a influenzare la rappresentazione anche negli strati più profondi, dove l'informazione è più semantica che geometrica.

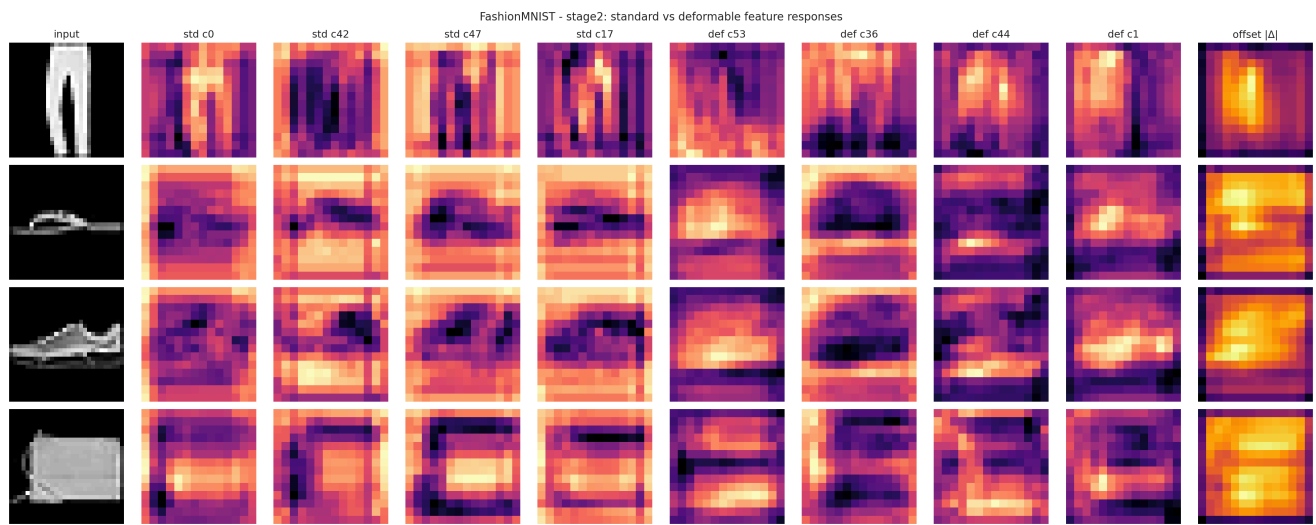
Nel complesso, le figure di MNIST sono coerenti con l'idea che la deformable convolution favorisca una migliore aderenza della risposta interna alla geometria dei digit, soprattutto quando la forma contiene stroke curvi o variazioni locali di orientamento.

FashionMNIST - stage 1, stage 2, stage 3

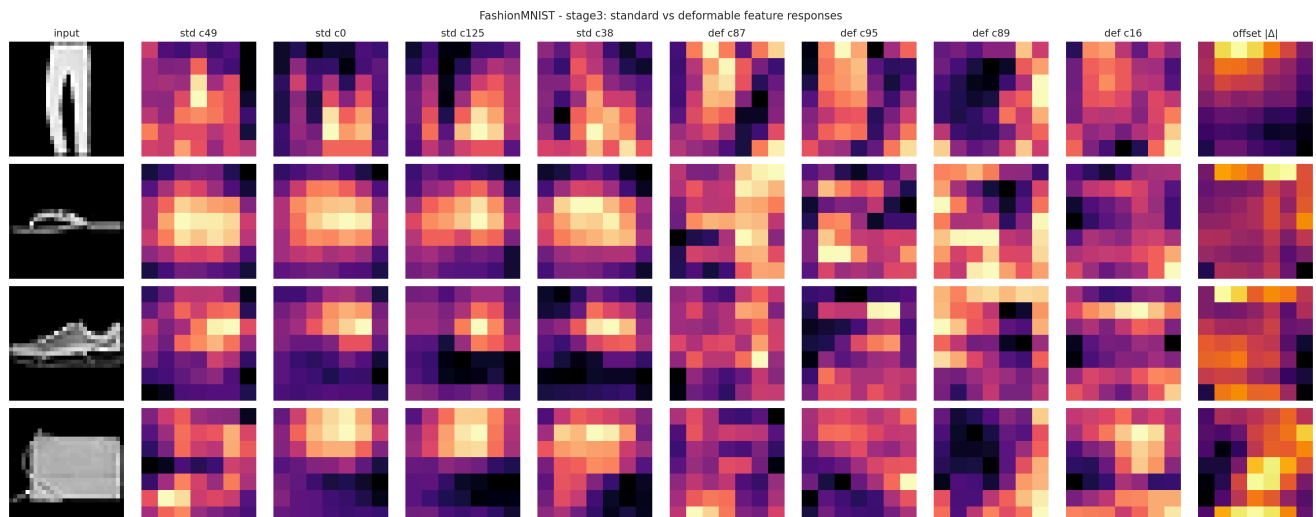
Su **FashionMNIST** l'interpretazione è ancora più interessante, perché gli oggetti hanno struttura più complessa dei digit e includono bordi, contorni, parti allungate e regioni con silhouette differenti.



Nel **primo stadio** la differenza più evidente riguarda la capacità di seguire i **contorni principali**. Nel caso del **trouser**, diverse feature deformabili sembrano allinearsi alle due bande verticali della forma; per **sandal** e **sneaker** si osservano attivazioni che enfatizzano la sagoma orizzontale o il profilo della calzatura; nel caso della **bag** emergono risposte concentrate sul bordo rettangolare e sulle discontinuità della silhouette. Le feature standard, pur catturando anch'esse parti rilevanti, appaiono in più casi più diffuse o meno adattate al profilo dell'oggetto.



Nel **secondo stadio** le mappe standard e deformabili mostrano differenze ancora più nette. Le attivazioni della DeformableCNN risultano spesso più localizzate su porzioni strutturalmente rilevanti dell'oggetto, mentre la heatmap degli offset indica che gli spostamenti medi tendono a concentrarsi nelle zone di transizione della forma, cioè dove il contorno e l'organizzazione spaziale dell'item portano più informazione discriminativa.



Nel **terzo stadio** la risoluzione ridotta rende le mappe più grossolane, ma il confronto resta utile: le feature deformabili continuano a mostrare risposte dipendenti dalla classe e non semplicemente uniformi o diffuse. In particolare, per classi con profilo molto diverso tra loro, come **bag** e **sneaker**, la rete deformabile conserva attivazioni che sembrano rispettare meglio la distribuzione globale della forma.

Interpretazione dell'esperimento 5

Questo quinto esperimento aggiunge un livello di evidenza qualitativa diverso rispetto all'esperimento 2. La visualizzazione degli offset mostra **come** cambia il campionamento; il confronto delle feature responses mostra invece **che effetto ha** questo cambiamento sulla rappresentazione interna.

La lettura più prudente e corretta è la seguente:

- la **StandardCNN** costruisce feature map coerenti con una griglia di campionamento rigida
- la **DeformableCNN** produce spesso attivazioni più aderenti a stroke, bordi, contorni e parti strutturali dell'oggetto
- l'effetto è più leggibile nei livelli intermedi, dove la rappresentazione è abbastanza astratta da essere informativa, ma non ancora troppo compressa da perdere leggibilità visiva

- le heatmap della magnitudine degli offset rafforzano l'idea che il modello deformabile concentri la deformazione soprattutto nelle regioni informative e non nello sfondo

Questa evidenza resta **qualitativa**: non dimostra da sola una causalità diretta tra singolo offset e singola decisione finale. Tuttavia è coerente sia con i risultati dell'esperimento 1, in cui la DeformableCNN ottiene accuracy superiore, sia con l'esperimento 3, in cui il vantaggio emerge soprattutto in presenza di variazioni geometriche.

Conclusione sul progetto

Il progetto realizza una pipeline completa, riproducibile e CPU-oriented che:

- implementa una **DeformableCNN** con tracciamento esplicito degli offset
- automatizza training, valutazione e salvataggio degli artefatti
- produce figure qualitative e heatmap per documentare il comportamento del meccanismo deformabile
- misura in modo controllato la robustezza a trasformazioni geometriche
- aggiunge toy examples esplicativi per convoluzione standard, convoluzione deformabile, pooling e interpolazione bilineare
- confronta direttamente le feature responses interne di layer standard e deformabili
- unifica l'intero workflow tramite `demo.py`

Con gli esperimenti 4 e 5 gli offset non vengono più mostrati soltanto come vettori o heatmap, ma vengono inseriti dentro una lettura più completa: prima si chiarisce visivamente il funzionamento dei layer su esempi sintetici, poi si osserva come quel meccanismo si rifletta nelle attivazioni reali dei modelli su input di test.

Il trade-off sperimentale più importante emerso resta l'**overhead computazionale su CPU** della deformable convolution: i guadagni in performance e robustezza osservati nel setup considerato sono accompagnati da tempi di training e inferenza sensibilmente superiori rispetto al baseline standard.